Part 3
효율적인 프로그래밍을 위한 중급 파이썬

13장 명령어 직접 만들어 사용하기: 사용자 정의 함수
14장 데이터와 함수를 한 단위로 묶어 재사용하기: 클래스와 객체
15장 프로그램 화면 구성하기: tkinter

Part 2에서는 지금까지 배운 파이썬 프로그래밍 요소를 활용해 실제 데이터로 데이터 분석 프로그램을 작성해 봤습니다. 단편적인 코드만 배우다가 전체 프로그램을 완성하니 좀 더 재밌지 않았나요?

Part 3에서는 사용자 정의 함수와 객체, 사용자 인터페이스(UI, User Interface)를 배워 Part 2에서 만든 프로젝트를 좀 더 효율적으로 사용할 수 있게 프로그램 구조를 개선해 보겠습니다. 사용자 정의 함수와 객체는 프로그래밍의 기초라고 하기는 어렵지만, 프로그래밍을 깊이 있게 배우려면 반드시 이해하고 넘어가야 합니다. 프로그램을 작성하는 사람의 시간과 노력을 획기적으로 단축하고, 이해하기 쉬운 코드를 작성하는 데 필수이기 때문이죠. 자, 그럼 사용자 정의 함수부터 배워 봅시다.

13장
명령어 적절 만들어 사용하기, 사용자 정의 함수

사용자 정의 함수를 본격적으로 배우기 전에 함수에 대해 알고 넘어갑시다. 함수라는 단어는 수학에서 시작된 개념입니다. 수학에 나오는 함수를 한번 생각해 봅시다.

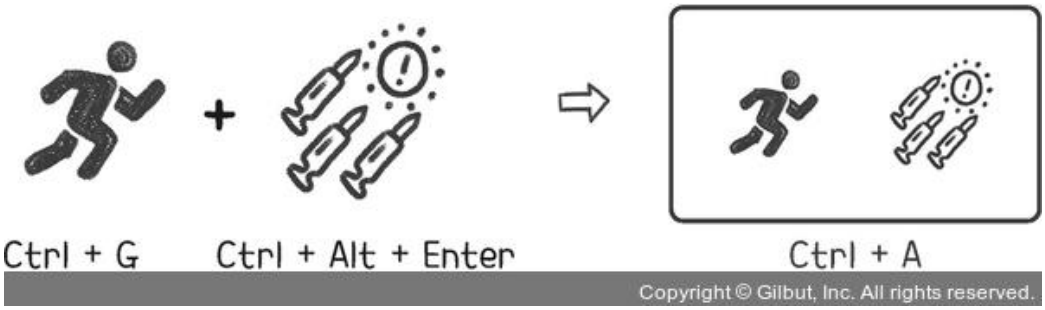
가령 $y = 10x$라는 식이 있을 때는 변수는 x, y, 변수 x에 따라 값이 정해지고 이렇게 변수 x에 따라 값이 정해질 때 y를 x의 함수라 하고 이러한 y를 $f(x)$라고 합니다. 파이썬 명령어인 컴퓨터가 바로 이해하는 컴퓨터 언어인 `print()` `input()` `plt.plot()` 등 우리가 사용한 명령어는 명령어(함수) 형태입니다. 명령어라는 단어를 들어 본 적이 없어도 사용한 모든 명령어는 정확히 표현하면 함수(function)입니다. 프로그래밍에서 함수는 특정 기능을 수행하는 코드 블록입니다.

프로그래밍 입문자가 함수를 만들지 않는 경우가 대부분입니다. 우리는 지금까지 파이썬에서 제공하는 함수만 사용해 왔기 때문입니다. `print()` `input()` 등이 내부 작업을 어떻게 하는지 모르고 그래도 잘 사용할 수 있는 함수를 내장 함수라고 합니다. 내장 함수와 반대되는 개념이 바로 사용자 정의 함수입니다. 사용자 정의 함수는 프로그램을 작성하는 사용자가직접 정의하는 함수를 말합니다. 정확히말하면 개발을 하다보면 자주 반복적으로 이루어지는 일이 있는데 이러한 일을 사용자가 직접 함수로 만들어 사용할 수 있는데 이것을 사용자 정의 함수라고 이야기합니다. 지금 당장 이해가 되지 않아도 괜찮습니다. 앞으로 배울 함수에 대해 사용자 정의 함수를 만들 수 있습니다.

13.1

단축키 역할을 하는 사용자 정의 함수

사용자 정의 함수는 우리가 익히 잘 아는 단축키 같은 개념입니다. 게임에서 공격하는 키와 앞으로 전진하는 키가 별도로 있을 때, 공격하고 전진하려면 각각의 키를 한 번씩, 총 두 번 눌러야 합니다. 단축키를 설정하면 편한 키 조합으로 전진하고 공격하기를 한 번에 할 수 있습니다.



▲ 그림 13-1 단축키 설정

프로그래밍에서도 단축키처럼 내장 함수를 조합해 다양한 기능을 한 번에 구현할 수 있습니다. 그럼 함께 사용자 정의 함수를 만들어 봅시다.

13.1.1 사용자 정의 함수 만들어 사용하기

게임에서 단축키를 사용하려면 먼저 단축키 조합을 만들어야 합니다. 그리고 단축키를 눌렀을 때 수행할 동작을 설정해야 단축키를 눌렀을 때 원하는 동작을 수행할 수 있습니다.

사용자 정의 함수도 마찬가지입니다. 먼저 함수명을 정하고, 함수가 수행할 기능을 지정해야 합니다. 그럼 첫 번째 사용자 정의 함수를 간단하게 만들어 봅시다.

```
def hi():
    print('hi!')
    print('My name is Python!')
```

조건문에서 `if`, 반복문에서 `for`를 사용하듯이 함수를 만들 때는 `def`를 사용합니다. `def` 다음에 만들고 싶은 함수의 이름(`hi`)을 적고 이름 다음에 소괄호(`()`)를, 마지막에 콜론(:)을 붙입니다. 그러고 나서 Enter를 누르면 조건문과 반복문에서처럼 자동으로 들여쓰기됩니다. 이 상태에서 함수가 수행할 기능을 작성하면 됩니다. 조건문에서 배웠듯이 들여쓰기는 소속을 시각적으로 표현하는 방법입니다. 앞의 코드는 `hi()`라는 사용자 정의 함수에 `print('hi!')`와 `print('My name is Python!')`이 속함을 보여 줍니다.

이렇게 함수명과 함께 함수가 어떤 기능을 수행할지 내용을 작성하는 것을 함수를 정의한다고 표현합니다. 즉, `hi()`라는 함수를 만들고, 이 함수는 `print('hi!')`와 `print('My name is Python!')`을 수행하는 함수라고 알려 주는 것이죠. 함수를 만들 때 가장 앞에 적었던 `def`도 정의한다는 의미의 영단어 define의 약어입니다.

형식

```
def 함수명():
    함수 내용
```

함수를 만들었으니 사용해 봅시다. 간단하게 함수명에 소괄호를 붙여 코드를 작성하면 실행됩니다.

```
def hi():
    print('hi!')
    print('My name is Python!')

hi()
```

```
실행결과
hi!
My name is Python!
```

`hi()` 함수를 사용했습니다. 다시 말해, 내부에 있는 `print('hi!')`와 `print('My name is Python!')`이 실행됐습니다. 단축키를 만든 후에는 단축키를 누르기만 하면 사용할 수 있죠. 사용자 정의 함수는 함수명에 소괄호를 붙여 코드에 넣는 것만으로도 함수를 사용할 수 있습니다. 이렇게 함수를 사용하는 것을 함수를 호출한다고 표현합니다. 호출은 상대방을 부르는 행위죠. 함수명으로 함수를 부른다고 생각하면 됩니다.

`hi()` 함수를 두 번 호출해 볼까요?

```
hi()
hi()
```

```
실행결과
hi!
My name is Python!
hi!
My name is Python!
```

`hi()` 함수의 내용이 두 번 실행되어 자기소개를 두 번 합니다. 이처럼 함수는 한 번만 정의하고, 그 이후에는 원하는 대로 호출해 사용할 수 있습니다.



▲ 그림 13-2 함수 정의와 함수 호출

한 가지 유의해야 할 점이 있습니다. 함수는 정의한 후, 호출해야 합니다. 함수를 만들지도 않았는데 사용할 수는 없죠. 다음 코드처럼 함수를 정의하기도 전에 호출하면 당연히 오류가 발생합니다.

```
hello()

def hello():
    print('hello!')
    print('My name is Python!')
```

```
실행결과
----> 1 hello()
NameError: name 'hello' is not defined
```

코드에서 `hello()` 함수를 정의하는 부분이 호출하는 부분보다 아래에 있죠. 프로그램은 위에서부터 아래로 실행되는 순차구조이기 때문에 `hello()`를 호출하기 전에 `hello()`가 정의돼야 합니다. 오류 내용에도 '`hello()`가 정의되지 않았다'고 나오네요.

```
def hello():
    print('hello!')
    print('My name is Python!')

hello()
```

```
실행결과
hello!
My name is Python!
```

`hello()` 함수를 정의하는 부분을 위로 올리고 그 후에 함수를 호출하니 오류가 발생하지 않습니다.

1분 퀴즈

1 다음 중 틀린 것을 고르세요.

코드 (가)

```
wakeup()
```

코드 (나)

```
def wakeup():
    print('일어날 시간입니다!')
    print('빨리 일어나세요!')
```

① 코드 (가)는 함수 `wakeup()`을 호출한다.

② 코드 (나)는 함수 `wakeup()`을 정의한다.

③ 코드 (나)의 `def`를 포함해야 함수가 제대로 정의된다.

④ 코드 (나) 아래에서 코드 (가)를 실행해야 오류가 발생하지 않는다.

⑤ 코드 (나) 아래에서 코드 (가)를 실행하면 '일어날 시간입니다!'만 출력되고 '빨리 일어나세요!'는 출력되지 않는다.

정답 및 해설

13.1.2 사용자 정의 함수의 입력과 출력

매개변수로 사용자 정의 함수에 데이터 전달하기

내장 함수를 사용할 때 소괄호 안에 데이터를 넣는 경우가 종종 있었습니다. 예를 들어 `input ('학번: ')`처럼 입력 안내문을 넣거나 `print('hello!')`처럼 출력할 문자열을 넣기도 하고, `len(s)`처럼 길이를 알고 싶은 데이터를 넣기도 합니다. 이처럼 내장 함수명 옆의 소괄호 안에 적절한 데이터를 넣어 주면 이 데이터를 활용해 내장 함수를 실행합니다.

사용자 정의 함수도 데이터를 받아 함수에서 사용할 수 있습니다. 함수 외부에서 데이터를 전달받아 저장하는 변수는 소괄호 안에 만들 수 있습니다. 다음과 같이 코드를 작성해 봅시다.

```
def hi(name):
    print('hi!')
    print('My name is', name, '!')
```

`hi()` 함수의 소괄호 안에 변수 `name`을 만들었습니다. 그리고 함수 내부에서 `name`으로 받은 데이터를 `print()` 함수를 사용해 출력합니다. `hi()`를 실행해 봅시다. `hi()`를 정의할 때 변수를 같이 정의했다면 변수에 전달할 데이터를 넣어 호출해야 합니다. 다음과 같이 코드를 작성해 봅시다.

```
hi(name = 'dragon')
```

```
실행결과
hi!
My name is dragon !
```

`name`에 `dragon`을 넣어 `hi()` 함수를 호출했습니다. 그러면 `hi()` 함수의 `print('My name is', name, '!')`가 실행될 때 `name`에는 `dragon`이 저장됐으므로 결과가 `My name is dragon !`으로 출력됩니다.

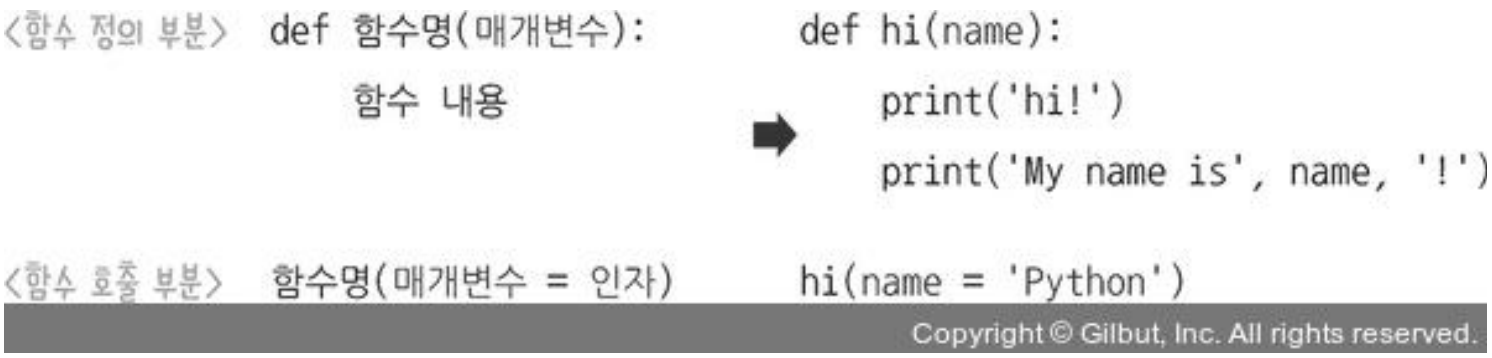
다른 데이터도 넣어 봅시다.

```
hi(name = 'Python')
hi(name = 2020)
```

```
실행결과
hi!
My name is Python !
hi!
My name is 2020 !
```

`name`에 `Python`을 넣으면 `My name is Python !`이, 숫자 `2020`을 넣으면 `My name is 2020 !`이 출력됩니다. `name`에 어떤 값을 전달하느냐에 따라 `hi()` 함수의 실행결과가 달라집니다.

이처럼 함수 외부에서 데이터를 전달받는 사용자 정의 함수의 변수를 매개변수(parameter)라고 합니다. 함수 외부와 함수 내부를 매개하는 변수라는 뜻입니다. 그리고 함수를 호출할 때, 매개변수에 넣는 데이터를 인자(argument)라고 합니다. 앞의 코드에서 매개변수는 `name`이고 인자는 `Python`과 `2020`입니다.



Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 13-3 매개변수와 인자

이제 오류에서 parameter나 argument라는 단어가 등장하면 parameter를 매개변수, argument를 인자로 해석하면 됩니다.

함수를 정의할 때 매개변수를 포함해 정의했는데 인자를 전달하지 않으면 어떻게 될까요? 다음 코드처럼 인자를 포함하지 않고 호출해 봅시다.

```
hi()
```

```
실행결과
----> 1 hi()
TypeError: hi() missing 1 required positional argument: 'name'
```

오류가 발생합니다. 오류 내용을 보면 `hi()` 함수에 필요한 위치 인자 1개를 누락했다고 나옵니다. `hi()` 함수를 호출할 때는 반드시 `name`이라는 항목이 필요한데 `name` 없이 호출했다는 뜻입니다. 위치 인자라는 용어는 뒤에서 다룹니다.

> TIP 사용자 정의 함수를 처음 배울 때 매개변수가 없는 `hi()` 함수를 정의했습니다. 그리고 이번에는 매개변수를 포함한 `hi()` 함수를 다시 정의했습니다. 한 프로그램 안에서 같은 이름의 함수를 여러 번 정의하게 되면 마지막에 정의한 함수로 동작하게 됩니다. 이는 변수와 같은 원리입니다. 같은 변수에 다른 값을 여러 번 저장할 수 있지만, 마지막으로 저장된 값만 변수에 기억되는 것과 같습니다.

이번에는 매개변수가 여러 개인 함수를 만들어 보겠습니다. 이름과 함께 직업까지 입력받는 `hi_2()` 함수를 다음처럼 정의하고 실행해 봅시다.

```
def hi_2(name, job):
    print('hi!')
    print('My name is', name, 'and My job is', job, '!')

hi_2(name = '고라니', job = 'programmer')
```

```
실행결과
hi!
My name is 고라니 and My job is programmer !
```

매개변수가 2개라면 각 매개변수에 각각 인자를 넣어 줍니다. `hi_2(name = '고라니', job = 'programmer')`는 매개변수 `name`에는 고라니, 매개변수 `job`에는 `programmer`를 인자로 넣어 `hi_2()` 함수를 호출한다는 뜻입니다.

매개변수의 순서를 바꿔 호출하면 어떻게 될까요? 이번에는 매개변수의 순서를 `job`, `name`으로 바꿔서 함수를 호출해 보겠습니다.

```
hi_2(job = 'programmer', name = '고라니')
```

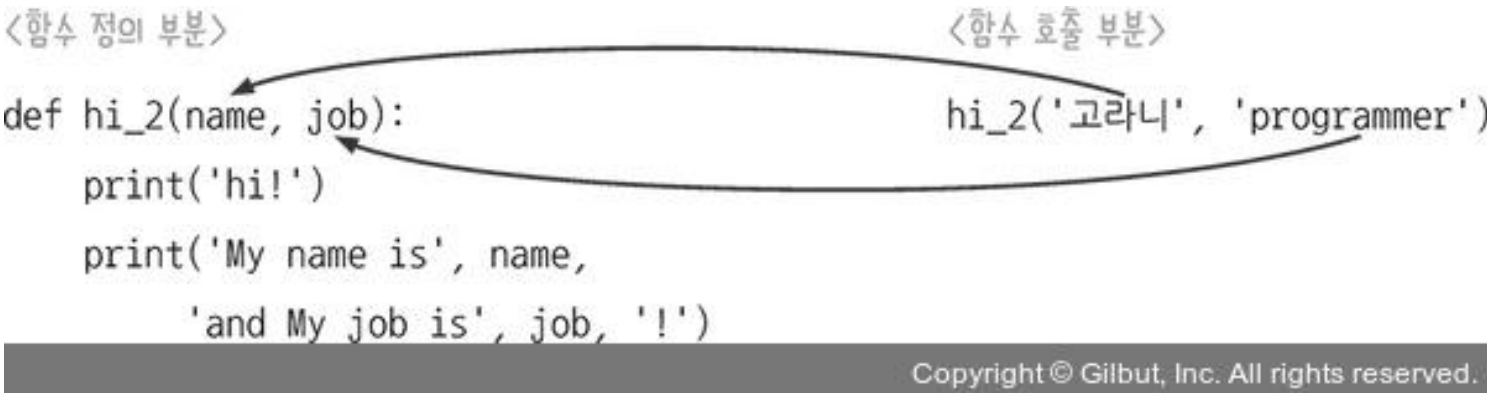
```
실행결과
hi!
My name is 고라니 and My job is programmer !
```

문제없이 출력됩니다. 매개변수와 인자를 직접 지정했기 때문에 순서와 상관없이 실행됩니다. 이렇게 `매개변수 = 인자` 방식으로 데이터를 매개변수에 직접 지정하는 방식을 키워드 인자라고 합니다. 그리고 함수를 호출할 때 매개변수를 생략하고 인자만 넣어 인자의 위치에 따라 매개변수와 매칭시키는 방법도 있습니다. 이를 위치 인자 방식이라고 합니다. 다음 코드를 실행해 봅시다.

```
hi_2('고라니', 'programmer')
```

```
실행결과
hi!
My name is 고라니 and My job is programmer !
```

매개변수 없이 인자만 넣어도 잘 실행됩니다. 처음에 들어오는 인자는 첫 번째 매개변수에, 두 번째로 들어오는 인자는 두 번째 매개변수에 자동으로 대응되기 때문입니다.



▲ 그림 13-4 위치 인자 대응 관계

이번에는 인자의 순서를 바꿔서 호출해 봅시다.

```
hi_2('programmer', '고라니')
```

```
실행결과
hi!
My name is programmer and My job is 고라니 !
```

인자의 순서를 바꾸니 결과가 달라집니다. 매개변수 없이 인자만 넣으면 매개변수의 순서대로 인자를 대응시키기 때문에 이런 현상이 발생합니다. 이처럼 함수를 호출할 때 매개변수를 명시해도 되고 생략해도 됩니다. 다만, 매개변수의 순서대로 인자가 대응되므로 매개변수를 생략할 때는 인자의 순서에 유의해야 합니다.

NOTE 함수에서 많이 발생하는 오류

함수를 호출할 때 매개변수 개수와 인자 개수가 일치하지 않으면 오류가 발생합니다.

```
hi_2('고라니')
```

```
실행결과
----> 1 hi_2('고라니')
TypeError: hi_2() missing 1 required positional argument: 'job'
```

`TypeError`가 발생합니다. 오류 내용을 보면 `hi_2()` 함수에 필요한 위치 인자 하나가 없다고 나옵니다. 함수 정의에는 매개변수가 2개인데, 인자는 1개여서 그렇습니다. 앞으로 이런 오류가 발생하면 당황하지 말고 매개변수와 인자 개수를 확인하세요.

함수를 정의할 때 매개변수에 초깃값을 넣을 수도 있습니다. 다음 코드를 실행해 봅시다.

```
def hi(name = 'A'):
    print('hi!')
    print('My name is', name, '!')

hi()
hi('B')
hi(name = 'C')
```

```
실행결과
hi!
My name is A !
hi!
My name is B !
hi!
My name is C !
```

매개변수 `name`에 초깃값으로 `'A'`를 저장했습니다. 초깃값이 저장됐기 때문에 `hi()`처럼 함수 호출 시 인자가 없어도 오류가 발생하지 않고, `name`에 저장된 초깃값 `'A'`를 넣어 실행합니다. 함수 호출 시 인자가 입력되면 해당 인자로 매개변수의 값을 바꿉니다. 따라서 `hi('B')`처럼 인자를 넣어 실행하면 자동으로 `name`에 `'B'`가 저장됩니다.

초깃값을 특정하기 어렵다면 `None`을 사용합니다. `None`은 값이 비어 있음을 표현하는 파이썬의 데이터 타입입니다.

```
def hi(name = None):
    print('hi!')
    print('My name is', name, '!')

hi()
```

```
실행결과
hi!
My name is None !
```

`None`을 넣고 실행하면 결과에는 `None`이 그대로 출력됩니다. 초깃값을 설정하기 어려울 때 `None`을 자주 사용합니다.

return으로 함수 외부에 값 전달하기
두 수의 합을 구하는 함수를 만들어 보겠습니다.

```
def plus(num1, num2):
    result = num1 + num2

plus(486, 849)
```

```
실행결과
```

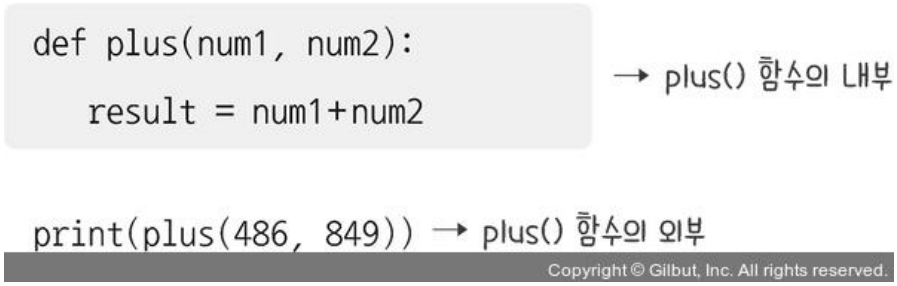
함수를 호출했는데 결과에 아무것도 출력되지 않습니다. 함수의 결과를 알려면 어떻게 해야 할까요? 결과가 출력되도록 `print()` 함수를 실행하면 될까요?

```
def plus(num1, num2):
    result = num1 + num2

print(plus(486, 849))
```

```
실행결과
None
```

`print()` 함수에 사용자 정의 함수 `plus()`를 넣어 실행해도 결과는 `None`이 나옵니다. 이는 함수를 호출해서 실행했으나 결괏값으로는 아무것도 받지 못했다는 의미죠. 함수 내부의 결괏값을 함수 외부에서 사용하려면 함수의 결괏값을 함수 외부로 전달해야 합니다. 여기서 함수 내부는 함수가 정의된 부분에서 들여쓰기 처리되어 함수에 소속되는 영역을 의미하고, 함수 외부는 함수에 소속된 부분 이외의 영역을 의미합니다.



▲ 그림 13-5 함수의 내부와 외부

코드를 실행하면 `plus()` 함수 내부에서 변수 `result`에 1335가 저장되지만, 함수 외부에서 `result`의 값에 접근할 수 있는 방법이 없습니다. 그래서 사용자 정의 함수 내부에서 만들어진 값을 함수 외부에 알려 주는 기능이 필요합니다. 이때 `return`을 사용합니다. `return` 옆에 값이나 변수를 작성하면 해당 값을 함수 외부에 전달합니다. 다음 코드를 실행해 봅시다.

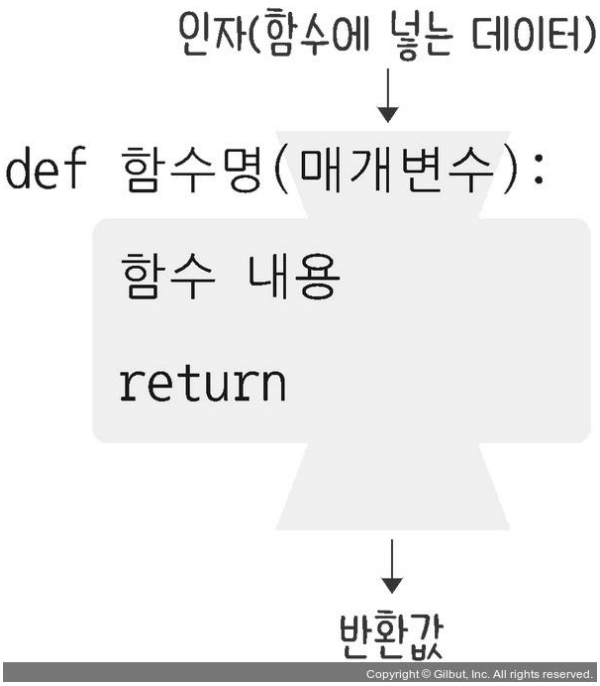
```
def plus(num1, num2):
    result = num1 + num2
    return result

print(plus(486, 849))
```

```
실행결과
1335
```

`print()` 함수 안의 `plus(486, 849)`가 실행되어 `plus()` 함수를 호출합니다. 호출된 `plus()` 함수는 486과 849를 인자로 받아 더합니다. 그런 다음 `num1 + num2`의 계산 결과인 1335를 `result` 변수에 저장합니다. `return result`는 변수 `result`에 저장된 값을 돌려준다는 의미이므로 `plus()` 함수를 호출한 곳으로 값 1335를 전달합니다. 따라서 `plus(486, 849)`를 실행한 결과는 1335가 됩니다. 이를 `print()` 함수로 출력하므로 실행결과로 1335가 출력됩니다.

return은 우리말로 '돌려주다, 반환하다'는 뜻입니다. 그래서 함수에 있는 `return`을 '함수의 결과를 돌려준다/반환한다'고 표현하며, 함수의 결과로 반환되는 결괏값을 반환값이라고 합니다.



▲ 그림 13-6 반환값

함수 외부에 `return`으로 결괏값을 돌려주고 나면 함수의 목적을 다했기 때문에 함수는 바로 종료됩니다. `return`은 결괏값을 반환하고 함수를 종료하므로 `return` 아래에 다른 명령어를 추가해도 실행되지 않습니다.

```
def plus(num1, num2):
    result = num1 + num2
    return result
    print(num1, '+', num2, '=', num1 + num2)

print(plus(486, 849))
```

실행결과

```
1335
```

`return`이 실행되는 즉시 `plus()` 함수가 종료되므로 `print(num1, '+', num2, '=', num1 + num2)` 코드는 실행되지 않습니다.

두 수를 입력받아 합을 구하는 프로그램을 작성해 봅시다. `plus()` 함수를 활용하는 간단한 프로그램이니 직접 작성해 본 후 책에 나온 코드를 보세요.

```
def plus(num1, num2):
    result = num1 + num2
    return result

a, b = int(input('첫 번째 숫자 : ')), int(input('두 번째 숫자 : '))
sum_ab = plus(a, b)
print('두 수의 합 :', sum_ab)
```

```
실행결과
첫 번째 숫자 : 379
두 번째 숫자 : 184
두 수의 합 : 563
```

함수를 정의해야 호출할 수 있으므로 가장 먼저 `plus()` 함수를 정의합니다. 그리고 함수 외부에서 숫자 2개를 입력받아 각각 변수 `a`, `b`에 저장합니다. `a`, `b` 변수를 인자로 `plus()` 함수를 호출해 두 수의 합을 구한 후 반환값을 출력합니다. 이처럼 함수의 인자로 데이터를 직접 넣지 않고 변수를 넣을 수도 있습니다. 사실 실제 프로그래밍에서는 인자로 데이터 자체를 넣기보다 변수를 넣는 경우가 훨씬 많습니다.

이번에는 환산점수를 계산하는 `calculate_score()` 함수를 작성해 보겠습니다. 환산점수는 100점 만점에 국영수 반영비율이 각각 30%, 30%, 40%로 가정합니다. 환산점수가 80점이 넘으면 합격, 80점 이하면 탈락이라고 안내합니다. 예를 들어, 한 학생의 국어 점수 99점, 영어 점수 80점, 수학 점수 72점일 때 어떤 값을 반환하는지 확인해 봅시다. 지금까지 함수를 잘 이해했다면 스스로 코드를 작성하고 책을 확인해 보세요.

```
def calculate_score(kor, eng, mat):
    score = kor * 0.3 + eng * 0.3 + mat * 0.4
    return score

student1 = calculate_score(99, 80, 72)
if student1 > 80:
    print('합격입니다! 점수는', student1)
else:
    print('탈락입니다. 점수는', student1)
```

```
실행결과
합격입니다! 점수는 82.5
```

`calculate_score()` 함수는 점수 3개를 전달받아 하나의 환산점수를 돌려줘야 합니다. 따라서 매개변수는 국어 점수 `kor`, 영어 점수 `eng`, 수학 점수 `mat`으로 총 3개가 필요합니다. 그리고 함수 내부에서 각 과목의 반영비율에 맞게 환산점수를 구하는 과정이 필요합니다. 이렇게 구한 최종 환산점수를 `score`에 저장합니다. 그리고 결괏값이 저장된 `score`를 반환값으로 돌려주고 함수를 종료합니다.

함수 외부에서는 각 과목 점수를 인자로 함수를 호출해 환산점수를 구합니다. 그리고 함수의 반환값을 `student1`에 저장하고 이 값이 80을 초과하면 합격, 그렇지 않으면 탈락이라고 안내합니다.

사용자 정의 함수를 만들 때 여러 개의 인자를 받고 싶다면 받고 싶은 인자의 개수만큼 매개변수를 설정하면 됩니다. 그렇다면 결괏값도 여러 개를 반환할 수 있을까요? 물론, 가능합니다. 각 반환값을 콤마(,)로 구분해 `return`과 연결하면 됩니다. `return a, b`처럼요.

`calculate_score()` 함수를 수정해 환산점수와 합격 여부를 한 번에 반환하는 함수를 만들어 봅시다.

```
def calculate_score_pass(kor, eng, mat):
    score = kor * 0.3 + eng * 0.3 + mat * 0.4
    if score > 80:
        pass_yn = 'yes'
    else:
        pass_yn = 'no'
    return score, pass_yn

stu1_score, stu1_pass = calculate_score_pass(99, 80, 72)
print('환산점수 :', stu1_score, '합격 여부 :', stu1_pass)
```

```
실행결과
환산점수 : 82.5 합격 여부 : yes
```

합격 여부를 저장하는 변수 `pass_yn`을 추가하고, 80점을 기준으로 조건 충족 여부에 따라 `pass_yn`에 `yes` 또는 `no`를 저장합니다. 그리고 결괏값 반환 부분을 `return score`, `pass_yn`으로 변경해 2개의 값을 반환하게 합니다. `calculate_score_pass()` 함수의 반환값이 2개이므로 이를 저장하는 변수도 2개여야 합니다. 반환값이 환산점수, 합격 여부 순서이므로 `stu1_score`에는 환산점수가, `stu1_pass`에는 합격 여부가 저장됩니다. 이처럼 반환값도 여러 개가 될 수 있습니다.

NOTE 튜플

반환값이 여러 개인데, 반환값을 저장하는 변수는 하나라면 어떻게 될까요?

```
stu1 = calculate_score_pass(99, 80, 72)
print(stu1)
```

실행결과

```
(82.5, 'yes')
```

오류가 발생하지는 않습니다. 다만, 반환값이 소괄호(`()`)로 묶여 나옵니다. 이 책에서는 여러 데이터를 묶어 표현하는 방법으로 총 두 가지를 배웠습니다. 대괄호(`[]`)로 묶으면 리스트, 중괄호(`{}`)로 묶으면 딕셔너리였죠. 앞의 결과처럼 소괄호로 묶인 데이터 타입은 튜플(tuple)이라고 합니다. 튜플은 리스트와 거의 유사하지만, 제한점이 많아 프로그래밍에서는 리스트가 더 많이 쓰입니다. 여러 개의 반환값을 하나의 변수에 담게 되면 이처럼 결괏값이 튜플로 나타납니다.

1분 퀴즈

2 사용자 정의 함수 `odd_even()`은 숫자 하나를 전달받아 홀수인지, 짝수인지를 판단합니다. 다음 중 `odd_even()` 함수에 관한 설명으로 옳지 않은 것을 고르세요.

```
def odd_even(num):
    if num % 2 == 1:
        result = '홀수'
    else:
        result = '짝수'
    return result

odd_even(num = 26)
```

① `num`은 함수 외부로부터 데이터를 전달받는 매개변수다.

② `odd_even(num = 26)`에서 `26`을 인자라고 한다.

③ 프로그램을 실행하면 `result`에는 `'짝수'`가 저장된다.

④ `return`으로 반환하는 값은 하나만 설정할 수 있다.

⑤ `odd_even(num = 26)`은 `odd_even(26)`으로 바꿔도 된다.

정답 및 해설

13.1.3 활동범위가 다른 지역변수와 전역변수

지금까지 사용자 정의 함수를 정의하고 호출하는 방법을 배웠습니다. 이번에는 사용자 정의 함수를 사용할 때 오류가 가장 많이 발생하는 부분인 지역변수와 전역변수의 개념을 다뤄 보겠습니다.

특정 지역에서만 사용하는 지역변수
다음 코드를 실행해 봅시다.

```
def hi():
    name = input('이름은? ')
    print('hi!')
    print('My name is', name, '!')

print('자기소개를 시작합니다.')
hi()
print(name, '님 안녕하세요.')
```

```
실행결과
자기소개를 시작합니다.
이름은? 제임스
hi!
My name is 제임스 !
---------------------------------------------------------------------------
----> 8 print(name, '님 안녕하세요.')
NameError: name 'name' is not defined
```

`name` 변수가 정의되지 않았다며 `NameError` 오류가 발생합니다. 분명 코드에 `name` 변수가 있는데 왜 정의되지 않았다고 나올까요?

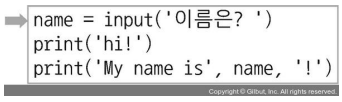
함수는 호출되기 전까지 불이 꺼져 내부가 보이지 않는다고 생각하면 됩니다. 함수 외부에서는 함수가 있다고는 인식하지만, 함수 내부가 어떻게 생겼는지는 알 수 없습니다. 함수가 호출되면 그제서야 불을 켜고 내부를 볼 수 있게 되죠. 함수가 종료되면 불이 다시 꺼집니다. 따라서 함수 내부에 정의된 변수는 호출되기 전까지 함수 외부에서는 보이지 않습니다. 코드를 보면 `name` 변수는 `hi()` 함수 내부에 정의되어 있습니다. 이렇게 함수 내부에 정의된 변수는 함수가 호출될 때만 사용할 수 있습니다. 코드의 실행 과정을 정리하면 다음과 같습니다.

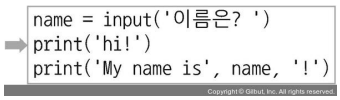
▼ 표 13-1 실행 과정

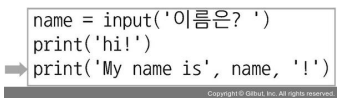
실행 순서	실행되는 명령어	설명

1	`def hi():`
	
	➡ `print('자기소개를 시작합니다.')`
	`hi()`
	`print(name, '님 안녕하세요.')`
		`hi()` 함수 내부가 보이지 않음

2	`def hi():`
	
	`print('자기소개를 시작합니다.')`
	➡ `hi()`
	`print(name, '님 안녕하세요.')`
		`hi()` 함수 호출

3	`def hi():`
	 
	`print('자기소개를 시작합니다.')`
	➡ `hi()`
	`print(name, '님 안녕하세요.')`
		`hi()` 함수 내부가 보이고, `hi()` 함수의 첫 번째 명령어 실행

4	`def hi():`
	 
	`print('자기소개를 시작합니다.')`
	➡ `hi()`
	`print(name, '님 안녕하세요.')`
		`hi()` 함수의 두 번째 명령어 실행

5	`def hi():`
	 
	`print('자기소개를 시작합니다.')`
	➡ `hi()`
	`print(name, '님 안녕하세요.')`
		`hi()` 함수의 의 세 번째 명령어 실행
		`hi()` 함수 종료

6	`def hi():` 	`hi()` 함수가 종료되며 내부가 다시 보이지 않음 다음 명령어 실행

Copyright © Gilbut, Inc. All rights reserved.

```
print('자기소개를 시작합니다.')
hi()
print(name, '님 안녕하세요.')
```

함수는 함수 내부의 세부적인 기능을 외부에는 숨깁니다. 그래프를 그리기 위해 `plt.plot()`을 사용하지만 그 내부가 어떤 구조인지 알지 못하는 것처럼 말이죠. `plt.plot()`으로 그래프를 그릴 데이터와 옵션을 넣을 뿐입니다. 사용자 정의 함수도 마찬가지입니다. 함수 내부에서 사용하는 변수를 함수 외부에서 알 수 없습니다. 이처럼 프로그램의 특정 지역(여기서는 함수 내부)에서만 사용하는 변수를 지역변수라고 합니다. 지역변수는 정해진 지역을 벗어나면 사용할 수 없습니다.

앞의 코드에서 `name`은 지역변수입니다. `name`에 저장된 값을 함수 외부에서 사용하고 싶다면 다음과 같이 `return`으로 반환해야 합니다.

```
def hi():
    name = input('이름은? ')
    print('hi!')
    print('My name is', name, '!')
    return name

print('자기소개를 시작합니다.')
person = hi()
print(person, '님 안녕하세요.')
```

```
실행결과
자기소개를 시작합니다.
이름은? 제임스
hi!
My name is 제임스 !
제임스 님 안녕하세요.
```

모든 지역에서 사용하는 전역변수
지역변수와 반대되는 개념으로 전역변수가 있습니다. 이름에서도 알 수 있듯이 전역변수는 프로그램 전체(전역)에서 사용되는 변수입니다. 따라서 전역변수는 사용자 정의 함수 내부가 아니라 외부에 정의된 변수입니다.

다음 코드를 봅시다. `func1()` 함수가 실행될 때마다 `a` 변수는 2만큼 증가하고, `b` 변수는 2배로 증가합니다. 이 코드에서 `a`와 `b`는 지역변수일까요? 전역변수일까요?

```
def func1():
    a += 2
    b *= 2
    print(a, b)

a, b = 5, 10 # a, b 변수 정의
func1()
```

이 코드에는 지역변수가 없습니다. `func1()` 함수 내부에 정의한 변수가 없기 때문이죠. 하지만 전역변수는 2개 있습니다. 바로 `a`와 `b`입니다. `a`와 `b`는 함수 외부의 프로그램이 진행되는 부분에 `a, b = 5, 10`으로 정의되어 있습니다. 이 코드를 실행하면 어떻게 될까요? 전역변수 `a`에 2를 더하고 `b`에 2를 곱할까요? 코드를 실행해 봅시다.

```
실행결과
      6 a, b = 5, 10
----> 7 func1()

      1 def func1():
----> 2 a += 2

UnboundLocalError: local variable 'a' referenced before assignment
```

오류가 발생합니다. 그리고 기존에는 오류를 표시하는 화살표가 하나였는데, 이번에는 2개고요. 첫 번째 화살표는 `func1()`에 있습니다. 함수 외부에서는 함수 내부를 볼 수 없기 때문에 첫 번째 화살표에서는 '오류 내용은 잘 모르겠지만 `func1()` 함수를 호출하는 부분에서 오류가 발생했다'고 알려 주기만 합니다. 두 번째 화살표는 호출된 `func1()` 함수의 어디에서 오류가 발생했는지를 구체적으로 알려 줍니다.

함수 정의가 가장 위에 있지만, 함수가 호출되기 전까지 실행되지 않습니다. 그래서 프로그램이 시작되면 가장 먼저 `a, b = 5, 10`이 실행되고, 두 번째로 `func1()` 함수가 호출됩니다. `func1()` 함수가 호출되고 나서야 `func1()`에 속한 명령어들이 실행되는 것이죠. 따라서 `func1()` 함수가 가장 위에 있지만, `func1()` 내부의 명령어들은 세 번째부터 실행됩니다.

▼ 표 13-2 실행 과정

실행 순서	실행되는 명령어

1	`def func1():`
	
	➡ `a, b = 5, 10`
	`func1()`

2	`def func1():`
	
	`a, b = 5, 10`
	➡ `func1()`

3	`def func1():`
	➡ `a += 2`
	`b *= 2`
	`print(a, b)`
	
	`a, b = 5, 10`
	➡ `func1()`

오류 내용을 봅시다. 처음 보는 `UnboundLocalError`입니다. Unbound는 '매여 있지 않은', '해방된'이라는 뜻이고, Local은 지역을 뜻합니다. 즉, 지역을 벗어난 오류란 의미죠. 지역변수는 특정 지역에서만 사용한다고 했습니다. 코드에서 변수 `a`와 `b`는 함수 외부에 정의되어 있으므로 전역변수입니다. 이런 전역변수의 값을 함수 안에서 수정하려고 해서(`a += 2`) 오류가 발생한 겁니다.

`a`, `b`의 값을 수정하는 부분을 주석 처리한 후 다시 실행해 봅시다.

```
def func1():
   # a += 2
   # b *= 2
    print(a, b)

a, b = 5, 10
func1()
```

```
실행결과
5 10
```

오류가 발생하지 않습니다. 무슨 차이일까요? 함수 내부에서는 전역변수의 값을 변경할 수 없고 읽을 수만 있습니다. 그래서 `a += 2, b *= 2`를 주석 처리하고 실행하면 `print(a, b)`는 실행됩니다.

정리하면, 지역변수는 함수 내부에서만 값을 읽고 변경할 수 있습니다. 전역변수는 프로그램 전역에서 값을 읽을 수 있지만, 함수 내부에서 값을 변경할 수 없습니다.

그런데 함수 내부에서 전역변수의 값을 변경해야 하는 경우가 있습니다. 이때는 함수 내부에서 전역변수를 사용하겠다고 `global`을 붙여 명시하면 값을 변경할 수 있습니다.

```
def func1():
    global a, b
    a += 2
    b *= 2
    print(a, b)

a, b = 5, 10
func1()
```

```
실행결과
7 20
```

오류 없이 실행됩니다. 결과를 보면 `a`는 2만큼 증가하고, `b`는 2배가 됐습니다. 전역변수의 값을 함수 내부에서 변경하는 데 성공했다는 의미죠.

TIP 함수 내부에서 전역변수의 값을 변경하는 대표적인 경우는 공통 데이터에 접근해야 할 때입니다. 마트에서 물건을 들여오고, 판매하는 과정을 생각해 봅시다. 물건 개수를 변수 `count`에 저장한다고 하겠습니다. 재고를 파악하는 함수는 `inventory()`이며, 물건을 들여오면 함수 내부에서 물건 개수인 `count`를 증가시킵니다. 판매하는 함수는 `sold()`이고, 물건이 팔릴 때마다 팔린 개수만큼 물건 개수인 `count`를 감소시킵니다. `count`는 `inventory()`와 `sold()` 모두 수정할 수 있어야 합니다. 이 경우 `count`를 전역변수로 사용하고 두 함수 내부에서는 `global` 키워드를 사용하여 `count` 값을 변경하면 됩니다.

이번에는 `func1()` 함수를 5번 반복해서 실행해 봅시다.

```
def func1():
    global a, b
    a += 2
    b *= 2
    print(a, b)

a, b = 5, 10
for i in range(5):
    func1()
```

실행결과

```
7 20
9 40
11 80
13 160
15 320
```

`a`는 5에서 시작해 7, 9, 11, 13, 15로 2씩 증가하고, `b`는 10에서 시작해 20, 40, 80, 160, 320으로 두 배씩 커집니다.

지금까지 살펴본 지역변수와 전역변수의 사용 범위를 정리하면 다음과 같습니다.

▼ 표 13-3 지역변수와 전역변수의 범위

변수	함수 내부	함수 외부
지역변수	값 읽기 O 값 변경 O	값 읽기 X 값 변경 X
전역변수	값 읽기 O 값 변경 X(`global` 사용 시 값 변경 O)	값 읽기 O 값 변경 O

지역변수와 전역변수 비교

지역변수와 전역변수의 차이를 프로그램을 작성해 알아봅시다.

상점이나 벌점을 주는 `reward_penalty()` 함수를 정의하고, 이 함수를 호출해 현재 점수에 상점 5점을 추가해 보겠습니다. 프로그램을 다음과 같이 작성합니다.

```
def reward_penalty(kind, points):
    print('현재 점수 :', current_point)
    if kind == '상점':
        current_point += points
    elif kind == '벌점':
        current_point -= points
    return current_point

current_point = 5
reward_penalty('상점', 5)
print('최종 점수 :', current_point)
```

현재 점수는 `reward_penalty()` 함수에 영향을 받지 않는 점수이므로 함수 외부에 `current_point`라는 이름으로 변수를 정의했습니다. `reward_penalty()` 함수는 상점인지 벌점인지(`kind`)와 그에 따른 점수(`points`)를 받아서 실행합니다. 따라서 매개변수가 2개 필요합니다. 만약 `kind` 변수의 값이 '상점'이라면 현재 점수(`current_point`)에 추가 점수(`points`)를 더하고, `kind` 변수의 값이 '벌점'이라면 현재 점수에서 추가 점수만큼 뺍니다. 프로그램을 실행해 보면 오류가 발생합니다. 왜 오류가 발생할까요?

> 실행결과
>
> ```
>       9 current_point = 5
> ---> 10 reward_penalty('상점', 5)
>
>       1 def reward_penalty(kind, points):
> ----> 2     print('현재 점수 :', current_point)
>
> UnboundLocalError: local variable 'current_point' referenced before assignment
> ```

`current_point`는 함수 외부에 정의됐기 때문에 전역변수입니다. 그런데 함수 내부에서 값을 변경하려고 해서 `UnboundLocalError`가 발생합니다.

함수 내부에서 전역변수 `current_point`의 값을 변경하고 싶다면 앞서 배운 것처럼 `global`로 전역변수 사용을 명시하면 됩니다.

```
def reward_penalty(kind, points):
    global current_point
    print('현재 점수 :', current_point)
    if kind == '상점':
        current_point += points
        print('상점 입력 후 점수 :', current_point)
    elif kind == '벌점':
        current_point -= points
        print('벌점 입력 후 점수 :', current_point)
    return current_point

current_point = 5
reward_penalty('상점', 5)
print('최종 점수 :', current_point)
```

```
실행결과
현재 점수 : 5
상점 입력 후 점수 : 10
최종 점수 : 10
```

함수에서 지역변수와 전역변수의 사용 범위를 이해하는 것이 매우 중요합니다. 사용 범위가 다른 두 변수를 잘못 쓰면 오류가 발생하거나 의도하지 않은 값이 변경될 수 있기 때문이죠. 프로그램을 작성할 때 변수의 사용 범위를 한 번 더 생각해 보기 바랍니다.

1분 퀴즈

3 `eat()` 함수는 음식의 이름과 칼로리를 받아서 해당 칼로리를 소모하기 위해 걸어야 하는 시간을 알려 줍니다. 다음 프로그램을 보고 옳지 않은 설명을 고르세요.

```
total_calorie = 0
food = []

def eat(food_name, food_calorie):
    (가)
    # 1시간 걸으면 250칼로리 소모된다고 가정
    walking_hour = food_calorie / 250
    print(food_calorie, '칼로리인', food_name, '를 소모하려면',
          walking_hour, '시간을 걸어야 합니다!')
    total_calorie += food_calorie
    food.append(food_name)
    print('지금까지 드신 칼로리는', total_calorie, ', 음식은', food, '입니다.')
    return walking_hour

hamburger_walking = eat(food_name = '햄버거', food_calorie = 750)
chicken_walking = eat(food_name = '치킨', food_calorie = 1000)
```

① `total_calorie`와 `food`는 전역변수다.

② 함수 외부에서 `print(walking_hour)`를 실행하면 오류가 발생한다.

③ 프로그램을 실행하면 `total_calorie`에는 `750`, `food`에는 `['햄버거', '치킨']`이 저장된다.

④ (가)에 들어갈 코드는 `global total_calorie, food`다.

⑤ (가)를 채우지 않고 프로그램을 실행했을 때 오류가 발생하지 않으려면 `total_calorie += food_calorie`와 `food.append(food_name)`이 없어야 한다.

정답 및 해설

13.1.4 사용자 정의 함수로 프로그램 재구성하기

이번에는 앞에서 만든 프로그램을 함수를 활용한 방식으로 수정해 보겠습니다. 다음은 5장에서 `break`를 배울 때 작성한 구구단 게임입니다.

```
import random

print('구구단을 외자! 구구단을 외자!')
for i in range(20):
    a, b = random.randint(2, 9), random.randint(1, 9)
    answer = int(input(str(a) + 'x' + str(b) + '? '))

    if answer != a * b:
        print('땡! 정답은', a * b)
        break
```

함수를 활용해 프로그램을 재작성해 봅시다. 우선 함수로 묶을 부분을 정해야 합니다. 문제를 내고 정답을 입력받아 오답 여부를 반환하는 부분을 함수로 작성해 봅시다. 함수명은 `gugudan()`으로 하겠습니다. 숫자 2개는 함수를 호출할 때 인자로 받게 합니다.

```
import random

def gugudan(num1, num2):
    answer = int(input(str(num1) + 'x' + str(num2) + '? '))

    if answer == num1 * num2:
        return 0
    else:
        print('땡! 정답은', num1 * num2)
        return -1
```

숫자 2개를 받아 곱해야 하므로 매개변수도 2개가 필요합니다. 매개변수는 `num1`, `num2`를 사용합니다. 문제를 내고 사용자로부터 입력받은 값이 정답인 경우와 오답인 경우를 `if-else`로 나누어 작성합니다. 원래 코드와 달리 변수 `a`, `b`가 매개변수 `num1`, `num2`로 변경됐으므로 이 부분을 알맞게 변경해야 합니다. 그리고 정답과 오답 여부를 함수 외부에 알려 줘야 하므로 정답일 경우 `0`을, 오답일 경우 `-1`을 반환합니다.

TIP 프로그래밍에서는 관습적으로 `return 0`은 정상 종료, `return -1`은 오류라는 의미로 사용합니다.

앞에서 작성한 `gugudan()` 함수를 호출해 구구단 게임을 완성합시다. 원래 코드를 참고해 프로그램을 작성합니다.

```
import random

def gugudan(num1, num2):
    answer = int(input(str(num1) + 'x' + str(num2) + '? '))

    if answer == num1 * num2:
        return 0
    else:
        print('땡! 정답은', num1 * num2)
        return -1

print('구구단을 외자! 구구단을 외자!')
for i in range(20):
    a, b = random.randint(2, 9), random.randint(1, 9)
    if gugudan(a, b) != 0:
        break
```

```
실행결과
구구단을 외자! 구구단을 외자!
6x3? 18
2x2? 4
9x6? 4
땡! 정답은 54
```

원래 코드와 동일하게 구구단 문제를 내고 정답이 맞는지 판단하는 부분이 20번 반복되게 합니다. 반복문 내부에서는 임의의 두 수를 생성해 변수 `a`, `b`에 저장합니다. 그리고 변수 `a`, `b`를 `gugudan()`의 인자로 전달합니다. `gugudan()` 함수는 인자로 받은 변수 `a`, `b`의 값을 각각 매개변수 `num1`, `num2`에 넣어 곱합니다. 사용자가 입력한 정답이 곱한 값과 맞는지 틀린지에 따라 `0` 또는 `-1`을 반환합니다. `gugudan(a, b)`의 반환값이 `0`이면 정답이므로 다음 문제를 계속해서 냅니다. 반환값이 `-1`이면 정답이 아니므로 `break`로 문제 출제를 중단합니다.

이번에는 5장 Self Check에 있던 프로그램을, 함수를 활용해 발전시켜 보겠습니다. 이름과 횟수를 입력받고, 횟수만큼 이름을 반복해서 말하는 게임입니다.

```
print('팅 팅팅팅 탱 탱탱탱 팅팅 탱탱 프라이팬 놀이!')
for i in range(5):
    name = input('이름? ')
    cnt = int(input('횟수? '))
    print(name * cnt)
```

원래 이 게임은 참가자가 정해져 있고, 참가자 중에서 이름을 호명해야 합니다. 이름이 틀리거나 참가자가 아닐 때는 벌칙을 받게 되죠. 여기서는 추가로 횟수를 1부터 4까지로 제한하겠습니다. 따라서 음수나 5 이상의 숫자를 외쳐도 역시 벌칙을 받습니다. 5장에서 작성한 코드에는 벌칙을 받는 부분이 없었습니다. 실제로 벌칙을 코딩하기는 어려우니 이름과 횟수 중 하나라도 틀렸을 때 틀렸다는 것을 알려 주는 기능을 넣어 보겠습니다.

먼저 참가자가 필요하죠. 참가자는 다음처럼 리스트로 만들어 `players`에 넣어 줍니다. 그리고 프로그램 전역에서 사용해야 하므로 함수 외부에 정의합니다.

```
players = ['까치', '강아지', '토끼', '고양이', '참새']
```

다음으로 필요한 함수를 작성합니다. 프로그램에 필요한 기능은 크게 두 가지입니다. 참가자 이름과 횟수를 외쳤을 때 횟수만큼 이름을 반복해서 출력하는 기능과, 이름과 횟수 중 하나라도 틀렸을 때 틀렸다는 것을 알려 주는 기능입니다. 두 기능을 각각 `game_play()`와 `wrong_play()` 함수로 다음과 같이 구현합니다.

```
def game_play(name, cnt):
    print(name * cnt)

def wrong_play(name, cnt):
    if name not in players:
        print(name, '은/는 없는 이름입니다!')
        return -1
    if cnt < 1 or cnt > 4:
        print('1~4의 숫자만 가능합니다! 입력한 숫자', cnt)
        return -1
```

`game_play()` 함수는 2개의 매개변수로 이름과 횟수를 전달받아 횟수만큼 이름을 출력합니다. `wrong_play()` 함수 역시 2개의 매개변수로 이름과 횟수를 전달받습니다. 그리고 벌칙을 받아야 하는 상황일 때 즉, 입력받은 이름(`name`)이 참가자 리스트에 없거나 횟수(`cnt`)가 1~4의 숫자가 아닐 때는 -1을 반환하고 함수를 종료합니다. `players`는 전역변수지만, 함수 내부에서 `players`의 값은 변경이 없으므로 `UnboundLocalError`가 발생하지 않습니다.

함수 외부의 게임 진행 부분도 작성해 봅시다.

```
print('팅 팅팅팅 탱 탱탱탱 팅팅 탱탱 프라이팬 놀이')

for i in range(5):
    next_player, count = input('이름? '), int(input('횟수? '))
    check = wrong_play(next_player, count)
    if check == -1:
        break
    else:
        game_play(next_player, count)
```

```
실행결과
팅 팅팅팅 탱 탱탱탱 팅팅 탱탱 프라이팬 놀이
이름? 까치
횟수? 3
까치까치까치
이름? 강아지
횟수? 4
강아지강아지강아지강아지
이름? 냥이
횟수? 2
냥이 은/는 없는 이름입니다!
```

원래 코드처럼 최대 5번 반복되도록 반복문을 작성합니다. 반복할 때마다 사용자로부터 이름과 횟수를 입력받아 이름은 `next_player`, 횟수는 `count`에 저장합니다. 이 부분은 한 줄로 작성할 수 있습니다. 두 변수를 `wrong_play()` 함수의 인자로 넘겨 틀린 부분이 없는지 확인합니다. 틀린 부분이 있다면 결과로 `-1`이 반환됩니다. 함수 호출 결과는 변수 `check`에 저장합니다. `check`가 `-1`이면 게임을 중단하고, `-1`이 아니면 `next_player`와 `count` 변수를 인자로 `game_play()` 함수가 호출됩니다.

1분 퀴즈

4 다음은 상품 가격이 만 원 이상일 경우 25% 할인을 적용하고, 그렇지 않으면 할인을 적용하지 않는 프로그램입니다. 다음 프로그램을 보고 틀린 것을 고르세요.

```
def cal_price(price):
    discount = 0
    if price >= 10000:
        print('25% 할인이 적용됩니다.')
        discount = price * 0.75
    else:
        print('할인되지 않습니다.')
        discount = price
    return discount

current_price = int(input('현재 가격 : '))
discount_price = cal_price((가))
print('할인 가격 : ', discount_price)
```

① `cal_price()` 함수의 매개변수는 `price`고, 함수의 반환값은 `discount`다.

② 프로그램의 마지막 줄에 `print(discount)`를 추가해도 정상적으로 실행된다.

③ (가)에는 `current_price`가 들어간다.

④ `cal_price()` 함수를 호출할 때 인자를 넣지 않으면 오류가 발생한다.

⑤ `cal_price()` 함수 내부에서 `current_price`의 값을 변경하고 싶다면, 함수 내부의 첫 줄에 `global current_price`를 추가한다.

정답 및 해설

13.2

함수를 사용해 핫플레이스 프로젝트 작성하기

이번에는 핫플레이스 프로젝트를 함수로 재구성해 보겠습니다. 프로그램 실행 시 나타나는 결괏값이나 출력은 같지만, 코드를 더 직관적이고 구조화된 형태로 변경하는 것이죠. 결과가 달라지지 않는데, 왜 이런 시간과 노력을 들일까요?

첫 번째, 재사용성 때문입니다. 프로그램을 작성하다 보면 비슷하거나 같은 기능이 여러 군데서 필요한 경우가 많습니다. 함수는 한 번 만들어 놓으면 호출만으로 기능을 간단하게 실행할 수 있습니다. 그래서 자주 사용하는 기능을 함수로 만들어 놓으면 편리합니다.

두 번째, 직관성 때문입니다. 기능을 매번 세세하게 살펴보는 것보다 세부 기능은 숨기고 그 기능을 대표하는 함수명으로 표현하면 프로그램 흐름이 직관적으로 눈에 들어옵니다. 앞서 `gugudan()`이나 `game_play()`, `wrong_play()`를 사용해 재구성한 코드를 보면 함수 정의 부분을 제외한 프로그램 진행 부분은 함수를 활용하기 전보다 깔끔하게 정리됐습니다. 이처럼 한눈에 프로그램의 흐름을 파악할 수 있기 때문에 함수를 활용합니다.

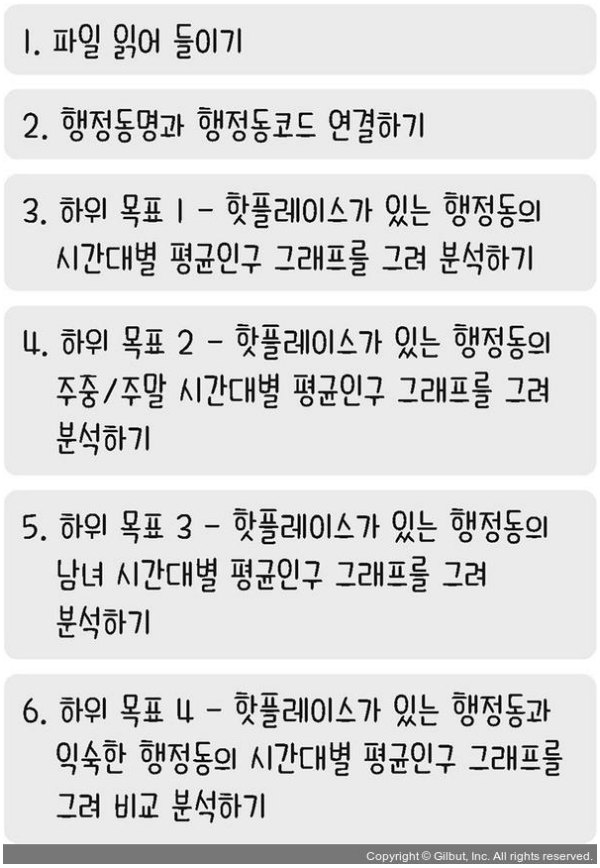
그럼 핫플레이스 프로젝트를 함수로 재구성해 봅시다.

13.2.1 함수로 재구성하기

함수를 활용하면 프로그램을 기능 단위로 나눠 관리할 수 있습니다. 프라이팬 놀이 게임에서 이름을 횟수만큼 반복해서 출력하는 기능과, 이름과 횟수 중 하나라도 틀렸을 때 틀렸다는 것을 알려 주는 기능을 각각 함수로 구현했죠.

그런데 함수를 사용해 프로그램을 작성할 때 유의해야 할 점이 있습니다. 함수 하나에 너무 많은 기능을 넣지 않아야 하는 것이죠. 함수에 너무 많은 기능을 포함하면 나중에 프로그램을 수정할 때 오히려 복잡해질 수도 있습니다. 함수에 포함된 여러 기능 중 하나만 골라서 사용해야 하거나 수정해야 할 때가 있기 때문입니다.

핫플레이스 프로젝트를 기능 단위로 쪼개 봅시다. 핫플레이스 프로젝트의 주제는 '핫플레이스가 덜 붐비는 시간대 찾기'죠. 이는 다음처럼 기능을 나눌 수 있습니다.

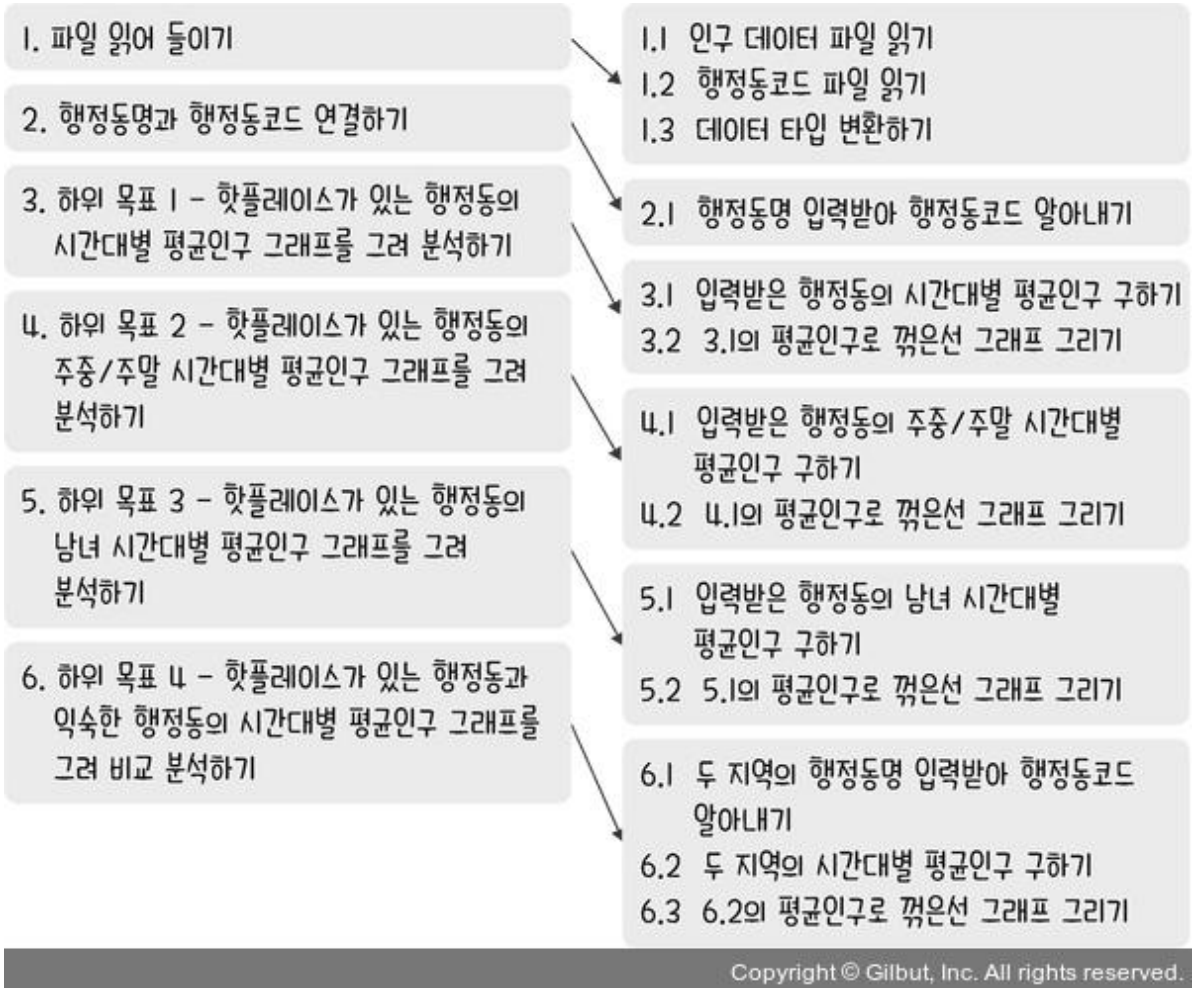


Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 13-7 기능별로 핫플레이스 프로젝트 나누기 1단계

이대로라면 총 6개로 나눌 수 있지만, '재사용'의 관점에서 조금 더 생각해 봅시다. 하위 목표에 공통으로 포함되는 더 작은 기능은 없을까요? 또는 여러 번 사용되는 기능은요?

앞의 6개 기능을 좀 더 세분하면 다음과 같습니다. 11장에서 작성한 알고리즘을 참고하면 쉽게 나눌 수 있어요.

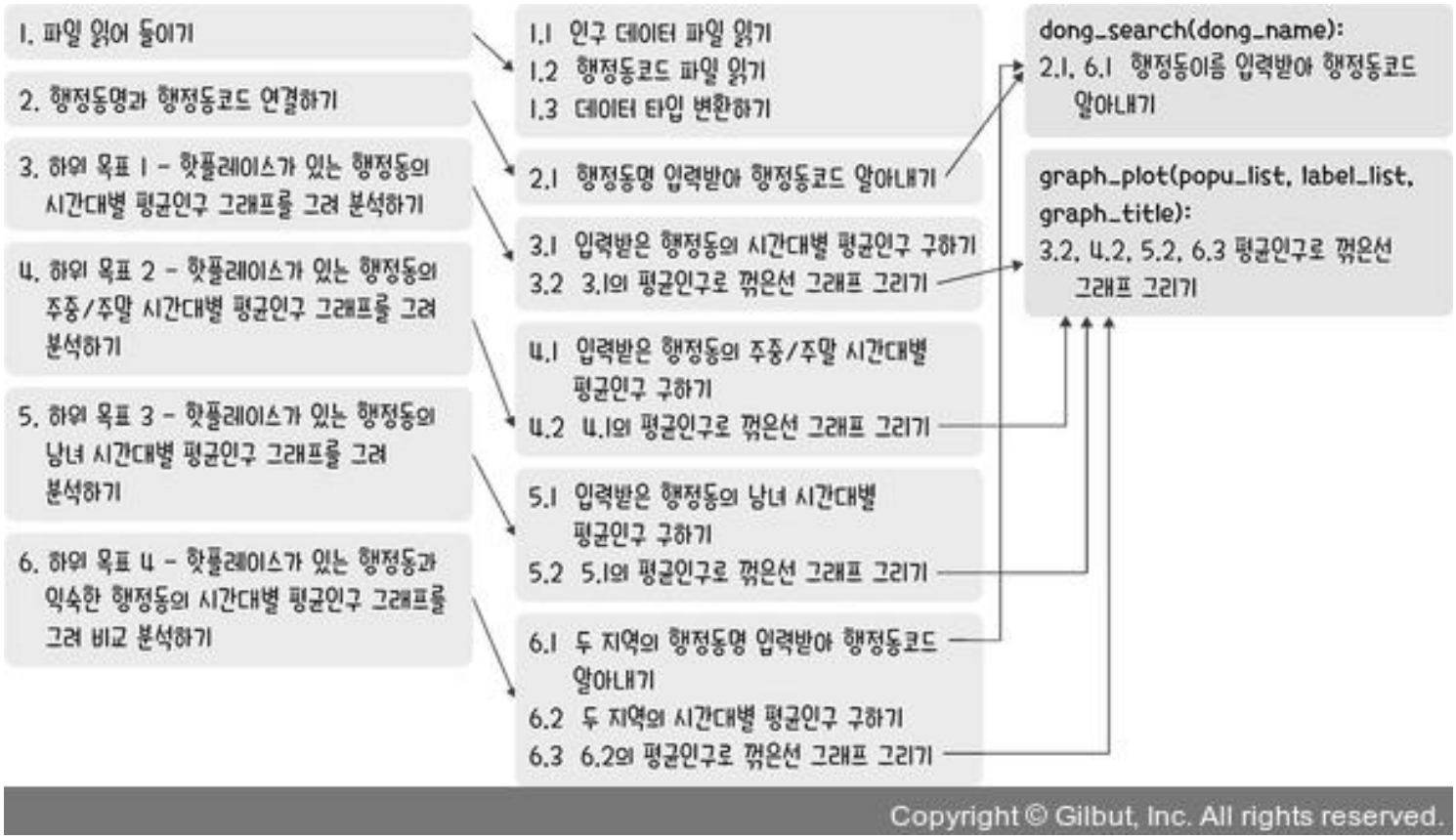


▲ 그림 13-8 기능별로 핫플레이스 프로젝트 나누기 2단계

11장에서는 데이터 특성이 잘 드러나는 그래프 형태로 데이터를 시각화했습니다. 그러나 이 장에서는 프로그램의 공통 부분을 찾는 것이 중요하므로 모든 결과를 꺾은선 그래프로만 그리겠습니다.

각 항목은 기능에 따라 2~3개의 하위 기능으로 다시 나눌 수 있습니다. 하위 기능을 나열하고 나니 공통 부분이 보입니다. '꺾은선 그래프 그리기'는 하위 목표에 모두 포함되어 있습니다. '꺾은선 그래프 그리기'를 함수로 만들면 되겠네요. 이 함수의 이름은 `graph_plot()`으로 합시다.

2.1의 '행정동명 입력받아 행정동코드 알아내기'가 6.1에서도 나옵니다. 이 역시 함수로 만들 수 있습니다. 함수의 이름은 `dong_search()`로 하겠습니다. 이를 반영하면 다음과 같습니다.



▲ 그림 13-9 기능별로 핫플레이스 프로젝트 나누기 3단계

11장에서는 동일한 기능의 코드가 반복되는 경우가 많았습니다. 그래프를 그리는 코드만 해도 5번이나 반복됩니다. 하지만 이제 재사용되는 기능을 함수로 정리하면 코드를 반복하는 대신 함수명만 호출하면 됩니다!

각 함수의 구조를 살펴봅시다. `dong_search()` 함수는 입력된 행정동명에 해당하는 행정동코드를 구하는 함수죠. 따라서 `dong_search()` 함수의 매개변수는 행정동명, 반환값은 행정동코드가 됩니다. 행정동명을 전달받으니 매개변수는 `dong_name`으로 하겠습니다.

`graph_plot()` 함수는 그래프를 그릴 인구 리스트가 필요합니다. 그리고 성별 인구나 주중/주말 인구처럼 두 가지 조건을 비교하는 그래프를 그릴 때는 어떤 그래프가 무엇을 의미하는지 알아야 합니다. 따라서 그래프에 적용할 라벨을 매개변수로 전달받아야 합니다. 그래프 제목도 있어야 하니 `graph_plot()`에는 매개변수가 총 3개 필요합니다. 인구 리스트는 `popu_list`로, 라벨명은 `label_list`로, 그래프 제목은 `graph_title`로 하겠습니다.

이제 나머지 부분을 함수로 바꿔 봅시다. 자주 재사용되는 부분은 아니지만, 함수로 구성하면 프로그램 흐름이 좀 더 정리됩니다.



▲ 그림 13-10 기능별로 핫플레이스 프로젝트 나누기 4단계

CSV 파일을 읽어 들이는 부분과 각 하위 목표에서 인구를 구하는 부분을 각각 함수로 구성합니다. CSV 파일을 읽어 들이는 부분은 항목이 3개지만, 각 항목이 개별적으로 재사용되지는 않아서 `file_open()`이라는 함수 하나로 구성합니다. 그리고 하위 목표 1~3은 함수명을 `analysis1~analysis3`으로 정하고 행정동명과 행정동코드를 매개변수 `dong_name`과 `dong_code`로 전달받습니다. 행정동코드는 하위 목표에 맞는 인구 리스트를 구성하는 데 사용하고, 행정동명은 그래프 이름을 만들 때 필요합니다. `analysis4` 함수는 다른 하위 목표 함수와 달리 매개변수가 4개입니다. 두 지역을 비교하기 때문에 행정동명과 행정동코드를 각각 2개씩 입력받아야 합니다. 따라서 `dong_name`과 `dong_code` 뒤에 숫자를 붙여 구분했습니다.

13.2.2 함수 작성하기

핫플레이스 프로젝트의 구조를 함수로 재구성했습니다. 이제 실제 코드로 옮겨 볼 차례입니다. 코드는 이미 11장에서 작성했고, 이 코드를 함수로 재구성하는 것이므로 앞서 작성한 코드를 보면서 진행하면 편합니다.

파일 읽기

첫 번째 함수인 `file_open()`부터 진행해 봅시다. 함수를 정의하는 형식에 따라 함수명을 적고 앞의 코드를 함수 안에 넣어 주면 됩니다. 바꿔 봅시다('11.2.1 데이터 파일일 읽고 행정동명과 행정동코드 연결하기'에 있는 변경 전 코드와 비교해 보세요).

file_open()

```
import csv

def file_open():
    # 1.1 인구 데이터 파일 읽기
    f = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8') # 또는 'eur-kr', 'cp949'
    data = csv.reader(f)
    next(data)
    data = list(data)

    # 1.2 행정동코드 파일 읽기
    f2 = open('dong_code.csv', encoding = 'cp949') # 또는 'eur-kr', 'utf8'
    code_data = csv.reader(f2)
    next(code_data)
    next(code_data)
    code_data = list(code_data)

    # 1.3 데이터 타입 변환하기 : 문자 --> 숫자
    # 인구 데이터 변환하기
    for row in data:
        for i in range(1, 32):
            if i <= 2:
                row[i] = int(row[i])
            else:
                row[i] = float(row[i])

    # 행정동코드 변환하기
    for row in code_data:
        row[1] = int(row[1])

    return data, code_data
```

파일을 열 때는 별도의 데이터를 전달받지 않으므로 매개변수는 없어도 됩니다. 함수 내부에서는 파일 내용을 리스트로 만들고 이를 변수 `data`와 `code_data`에 저장한 후 마지막에 반환합니다. 그리고 프로그램에서 `file_open()`을 다음과 같이 호출합니다.

함수 호출 부분

```
data, code_data = file_open()
```

`file_open()` 함수의 결과, 즉 반환값을 `data`와 `code_data`에 저장합니다. `file_open()` 내부에 있던 `data`와 `code_data`는 지역변수지만, `file_open()` 함수의 반환값을 저장하는 `data`와 `code_data`는 전역변수입니다. 이름은 같지만, 사용 범위가 전혀 다르니 헷갈리면 안 됩니다. 전역변수 `data`와 `code_data`는 다른 함수에서도 읽을 수 있는 변수입니다.

행정동명과 행정동코드 연결하기

그럼 두 번째 함수를 만들어 봅시다. 행정동명을 입력받으면 행정동코드를 알려 주는 `dong_search()` 함수입니다. `dong_search()` 함수는 행정동명을 입력받아 행정동코드를 반환합니다. 매개변수 `dong_name`으로 행정동명을 입력받고 함수 내부에서 찾은 행정동코드는 `code`에 저장하겠습니다(변경 전 코드는 '11.2.1 데이터 파일일 읽고 행정동명과 행정동코드 연결하기'에 있습니다).

dong_search()

```
def dong_search(dong_name):
    for row in code_data:
        if row[-1] == dong_name:
            code = row[1]
    print(dong_name, '-', code, '을(를) 분석합니>!')
    return code
```

프로그램에서 `dong_search()` 함수를 호출할 때는 인자로 행정동명을 넣어 호출하고, 반환값으로 받은 행정동코드를 저장하면 됩니다.

함수 호출 부분

```
data, code_data = file_open()

input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)
```

사용자에게 행정동명을 입력받아 `input_name`에 저장하고, `dong_search()` 함수를 호출할 때 `input_name`을 인자로 넣습니다. 함수 실행결과로 반환되는 값은 `input_code`에 저장합니다. 앞으로 진행할 인구 데이터 분석은 모두 `input_name`과 `input_code`에 저장된 지역을 기준으로 합니다.

그래프 그리기
다음 함수를 작성해 봅시다. 프로그램 실행 순서대로라면 하위 목표 1에 해당하는 부분(3.1)을 작성해야 합니다. 그런데 모든 하위 목표에서 공통으로 사용하는 기능이 있죠. 바로 꺾은선 그래프 그리기입니다. 그래프 그리기 함수가 먼저 정의되어야 이 함수를 각 하위 목표에서 호출할 수 있으므로 `graph_plot()` 함수부터 작성합니다.

하위 목표 1, 2, 4의 그래프를 먼저 봅시다. 이전에 작성한 코드는 다음과 같습니다.

```
import matplotlib.pyplot as plt

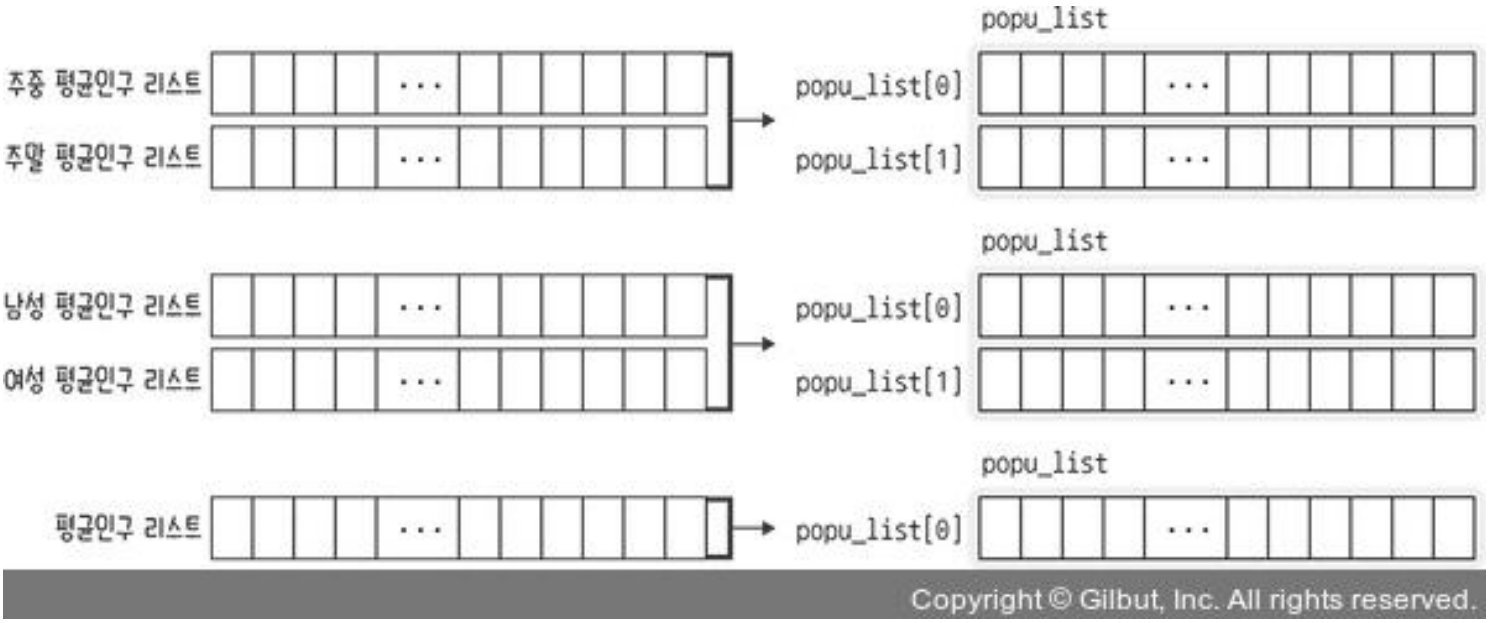
# 하위 목표 1의 그래프 그리기
plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + ' 시간대별 평균인구')
plt.plot(range(24), population, color='indigo')
plt.xticks(range(24), range(24))
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.show()

# 하위 목표 2의 그래프 그리기
plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + ' 주중/주말 시간대별 평균인구')
plt.plot(weekday, color='indigo', label='주중')
plt.plot(weekend, color='orangered', label='주말')
plt.legend()
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.xticks(range(24), range(24))
plt.show()

# 하위 목표 4의 그래프 그리기
plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + '과 ' + dong_name2 + ' 시간대별 평균인구 비교')
plt.plot(population, color='m', label=dong_name)
plt.plot(population2, color='orange', label=dong_name2)
plt.legend()
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.xticks(range(24), range(24))
plt.show()
```

글꼴, 그래프 제목, x축과 y축 이름 설정, x축 눈금 설정, 그래프 보여 주기가 공통으로 들어 있습니다. 다른 부분은 그래프를 그리는 `plt.plot()` 함수의 호출 횟수와 범례를 보여 주는 `plt.legend()` 함수입니다. 그래프를 하나만 그릴 때는 인구 리스트 하나를 전달받습니다. 그래프를 2개 그려야 할 때는 서로 다른 인구 리스트 2개를 전달받습니다. 또한, 전달받는 인구 리스트의 개수에 맞춰 `label` 옵션의 개수와 `plt.legend()` 함수의 호출도 달라집니다. 하위 목표 1에서는 그래프가 하나이므로 별도의 `label`이 없습니다. 따라서 `plt.legend()`도 없죠. 하지만 하위 목표 2와 4에서는 두 그래프를 구분하기 위한 `label`이 있고 `plt.legend()`도 호출합니다. 이처럼 바뀌는 부분이 여럿인데 매개변수 하나로 받을 수 있을까요?

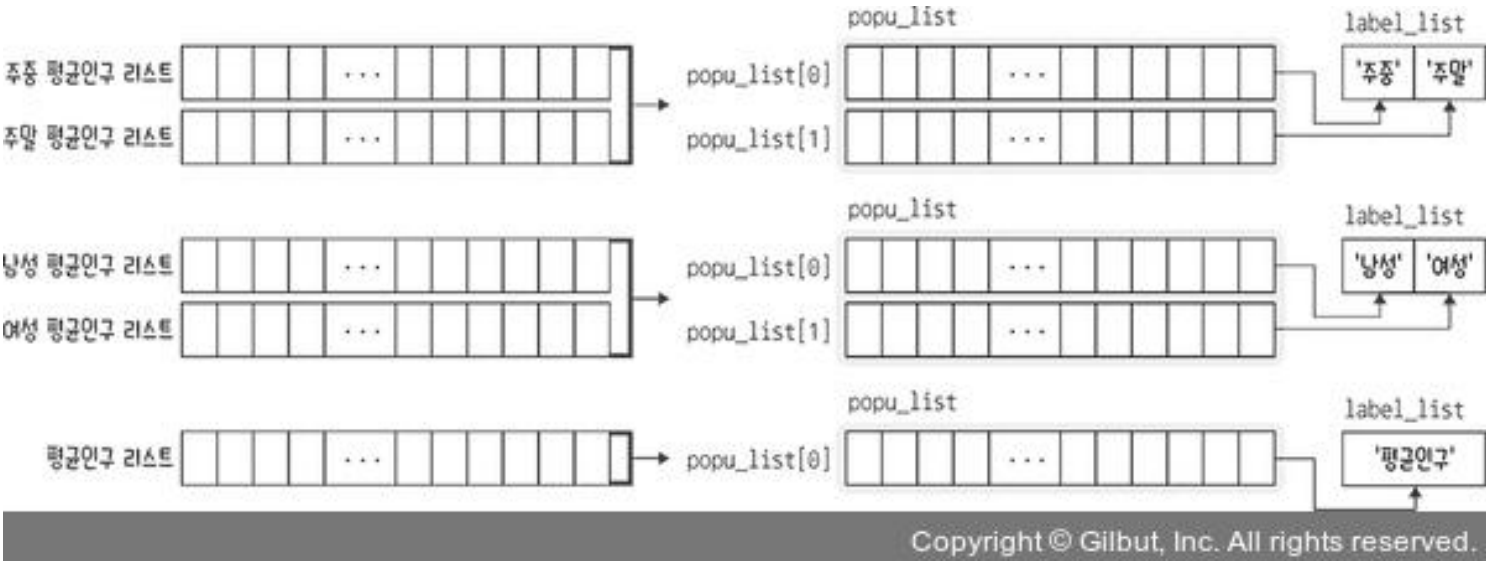
생각을 전환해 봅시다. 여러 개 리스트를 입력받아야 한다면 이 리스트를 묶어서 하나로 만들면 되지 않을까요? 일차원 리스트를 묶은 이차원 리스트로요.



▲ 그림 13-11 인구 리스트를 이차원 리스트로 만들기

그림처럼 주중 인구 리스트와 주말 인구 리스트를 합쳐 2 × 24 형태의 이차원 리스트로 만듭니다. 이차원 리스트의 이름을 `popu_list`라고 하면 `popu_list[0]`(`popu_list`의 첫 번째 행)은 주중 인구 리스트를, `popu_list[1]`(`popu_list`의 두 번째 행)은 주말 인구 리스트를 의미하게 됩니다. 남성과 여성 인구 리스트도 묶어서 이차원 리스트를 만들 수 있습니다. 이렇게 만든 이차원 리스트를 그래프를 그리는 `graph_plot()` 함수의 매개변수로 넣어 줍니다.

그래프 라벨도 리스트로 만듭니다. 그래프가 주중 인구, 주말 인구 2개라면 `label_list = ['주중', '주말']`로 리스트를 만들면 됩니다. 그리고 그래프 개수가 하나인 하위 목표 1은 `label_list = ['평균인구']`로 만듭니다.



▲ 그림 13-12 그래프 라벨을 이차원 리스트로 만들기

`popu_list`와 `label_list`가 인덱스별로 대응해 하나의 그래프를 그리게 됩니다. 주중 인구 그래프는 `plt.plot(popu_list[0], label=label_list[0])`으로, 주말 인구 그래프는 `plt.plot(popu_list[1], label=label_list[1])`로 그릴 수 있습니다. 코드를 보면 반복되죠. 달라지는 것은 리스트의 인덱스뿐입니다. 따라서 `plt.plot(popu_list[i], label=label_list[i])`를 반복문에 넣어 `i`가 0일 때부터 마지막 인덱스일 때까지 반복하면 됩니다.

그럼 `graph_plot()` 함수를 작성해 봅시다. 앞에서 원래 코드를 함수로 구성할 때 `graph_plot()`의 매개변수는 총 3개였습니다. 각각 인구 데이터, 그래프 라벨, 그래프 제목을 의미하는 `popu_list`, `label_list`, `graph_title`였죠. 프로그램에서 `graph_plot()` 함수를 호출할 때 `popu_list`에는 이차원 리스트를, `label_list`에는 일차원 리스트를, `graph_title`에는 문자를 입력하는 함수를 작성합니다.

graph_plot()

```
import matplotlib.pyplot as plt

def graph_plot(popu_list, label_list, graph_title):
    plt.rc('font', family='Malgun Gothic')
    plt.title(graph_title)
    for i in range(len(popu_list)):
        plt.plot(range(24), popu_list[i], label=label_list[i])
    plt.legend()
    plt.xlabel('시간대')
    plt.ylabel('평균인구수')
    plt.xticks(range(24), range(24))
    plt.show()
```

한글 글꼴을 설정하고, 전달받은 매개변수 `graph_title`로 그래프 제목을 넣었습니다. 그 다음 인구 데이터로 입력받은 `popu_list`의 첫 행부터 마지막 행까지 반복합니다. 그리고 반복문 내부에서 `popu_list`의 인덱스와 `label_list`의 인덱스를 맞춰 `plt.plot()`으로 그래프를 그립니다. 그래프 색상은 자동으로 설정되도록 따로 지정하지 않았습니다. 나머지 공통 부분을 추가해서 함수를 완성했습니다.

각 하위 목표에서는 `graph_plot()` 함수를 호출해 그래프를 그립니다. 예를 들어, 하위 목표 1 (3.1)에서는 시간대별 평균인구 리스트로 그래프를 그려야 하므로 마지막에 `graph_plot()` 함수를 호출합니다. 다른 하위 목표도 먼저 알맞은 인구 데이터를 도출한 후, 해당 데이터를 `graph_plot()` 함수에 매개변수로 넘겨 그래프를 그립니다.

> TIP 이 책에서는 최대 2개의 그래프를 동시에 그리는 경우만 다뤄서 굳이 이차원 리스트로 합치지 않아도 함수로 구현할 수 있습니다. 하지만 이렇게 여러 개의 인구 리스트를 이차원 리스트로 `graph_plot()`에 전달하면 인구 리스트가 3개 이상이라도 그래프를 그릴 수 있는 장점이 있습니다. 예를 들어 미성년, 청년, 중년, 장년, 노년의 평균인구 그래프를 동시에 그린다고 가정해 봅시다. 리스트가 5개 필요하죠. 이때 길이가 24인 일차원 리스트 5개를 하나로 묶은 이차원 리스트를 `graph_plot()`의 `popu_list`로 전달하면 반복문에서 자동으로 이차원 리스트의 길이를 계산해(이 경우에는 5가 됩니다) 5번의 `plt.plot()`을 수행합니다. `label_list`의 값을 자동으로 라벨로 넣으면서 말이죠. 동시에 여러 개의 그래프를 그리는 코드를 추가로 실습하면 그 의미를 더 정확하게 이해할 겁니다.

그럼 이제 4개의 하위 목표를 하나씩 함수로 구성해 봅시다.

하위 목표 1 - 시간대별 인구 분석하기

첫 번째 하위 목표는 시간대별 평균인구를 그래프로 그려 분석하는 것이었죠. 따라서 하위 목표 1의 함수명은 `analysis1()`으로 하고, 함수 외부에서 행정동명과 행정동코드를 `dong_name`과 `dong_code`로 입력받습니다. 이에 맞게 코드를 작성해 봅시다(변경 전 코드는 '11.2.2 하위목표 1-시간대별 인구 분석하기'에 있습니다).

```
def analysis1(dong_name, dong_code):
    # 3.1 입력된 행정동의 시간대별 평균인구 구하기
    population = [0 for i in range(24)]
    for row in data:
        if row[2] == dong_code:
            time, p = row[1], row[3]
            population[time] += p
    population = [p/31 for p in population]

    # 3.2 3.1에서 구한 평균인구 리스트로 꺾은선 그래프 그리기
    population = [population]
    labels = ['평균인구']
    title = dong_name + ' 시간대별 평균인구'
    graph_plot(popu_list = population, label_list = labels, graph_title = title)
```

시간대별 평균인구를 저장하는 `population` 리스트를 만드는 부분은 그대로 `analysis1()` 함수에 들어갑니다. 그 아래 그래프를 그리는 부분은 앞서 작성한 `graph_plot()` 함수를 호출하는 것으로 변경되죠. 이때 `graph_plot()`의 매개변수 `popu_list`, `label_list`, `graph_title`에 전달할 인자를 설정합니다.

인구 데이터를 받는 매개변수 `popu_list`는 `graph_plot()` 함수를 정의할 때 이차원 리스트로 받기로 했습니다. 그래서 일차원 리스트인 `population`을 인자로 넘길 때는 대괄호(`[]`)로 한 번 더 감싸서 이차원 리스트로 만들어야 합니다. 매개변수 `label_list`도 리스트로 받으므로 그래프 라벨인 `labels`도 일차원 리스트로 만듭니다. 그래프 제목은 기존대로 행정동명(`dong_name`)으로 그래프 제목을 만들어 `title` 변수에 저장합니다. 그리고 이 변수들을 `graph_plot()`의 인자로 넘겨 줍니다.

> TIP 그래프를 하나만 그리는데도 이차원 리스트로 만들어야 할까요? 결론부터 말하면 그렇습니다. 그렇지 않으면 오류가 발생합니다. 일차원 리스트를 넣으면 `graph_plot()` 함수의 반복문에서 반복 횟수는 24가 되므로(`len(popu_list`) 그래프를 24번 그리게 됩니다(물론 24개의 데이터가 없기 때문에 그려지지 않고 오류가 발생합니다). 그래서 꺾은선 그래프를 하나만 그리더라도 인구 리스트는 이차원 리스트 형태로 넘겨야 합니다.

프로그램에 `analysis1()` 함수를 호출하는 부분을 추가해 봅시다.

함수 호출 부분

```
data, code_data = file_open()

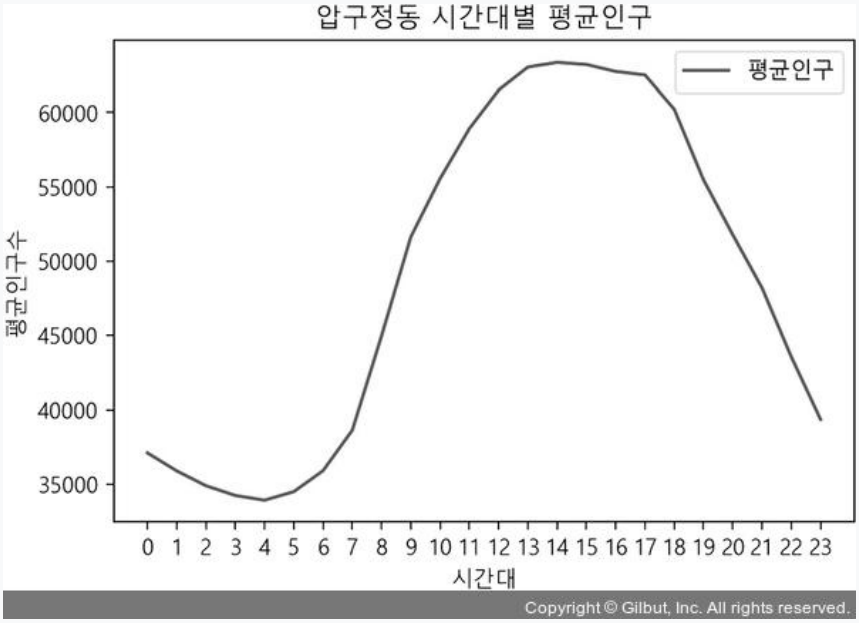
input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)

analysis1(dong_name = input_name, dong_code = input_code)
```

지금까지 작성한 함수와 함수 호출 부분을 모두 합쳐 실행해 보세요.

실행결과

```
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
압구정동 - 11680545 을(를) 분석합니다!
```



11장과 같은 모양의 그래프가 나타납니다. 범례만 추가됐을 뿐이죠. 함수를 활용하니 프로그램의 구조가 더 잘 보이지 않나요? 남은 부분도 함수로 재구성해 봅시다.

하위 목표 2 - 주중/주말 시간대별 인구 분석하기

하위 목표 2도 수정해 봅시다. 이 역시 그래프 그리는 부분은 `graph_plot()` 함수를 호출하는 것으로 대신하고, `graph_plot()` 함수에 넣을 인자를 만들어 주면 됩니다(변경 전 코드는 '11.2.3 하위 목표 2 - 주중/주말 시간대별 인구 분석하기'에 있습니다).

```
import datetime

def analysis2(dong_name, dong_code):
    # 4.1 입력된 행정동의 주중/주말 시간대별 평균인구 구하기
    weekday = [0 for i in range(24)]
    weekend = [0 for i in range(24)]

    for row in data:
        if row[2] == dong_code:
            time, p = row[1], row[3]
            year, mon, day = int(row[0][:4]), int(row[0][4:6]), int(row[0][6:])
            num = datetime.date(year, mon, day).weekday()
            if num < 5:
                weekday[time] += p
            else:
                weekend[time] += p

    weekday_cnt, weekend_cnt = 0, 0
    for i in range(1, 32):
        if datetime.date(2019, 12, i).weekday() < 5:
            weekday_cnt += 1
        else:
            weekend_cnt += 1

    weekday = [w/weekday_cnt for w in weekday]
    weekend = [w/weekend_cnt for w in weekend]

    # 4.2 4.1에서 구한 평균인구 리스트로 꺾은선 그래프 그리기
    data_set = [weekday, weekend]
    labels = ['주중', '주말']
    title = dong_name + ' 주중/주말 시간대별 평균인구'
    graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

역시 그래프 2개를 겹쳐 그리기 때문에 일차원 인구 리스트 2개를 이차원 리스트로 만들어야 합니다. 첫 번째 행이 `weekday`, 두 번째 행이 `weekend` 리스트가 되는 이차원 리스트 `data_set`을 만듭니다. 그래프에 붙일 라벨과 그래프 제목도 변수로 만든 후 `graph_plot()` 함수의 인자로 넘깁니다.

마지막으로 프로그램에 `analysis2()` 함수를 호출하는 부분을 추가합니다.

함수 호출 부분

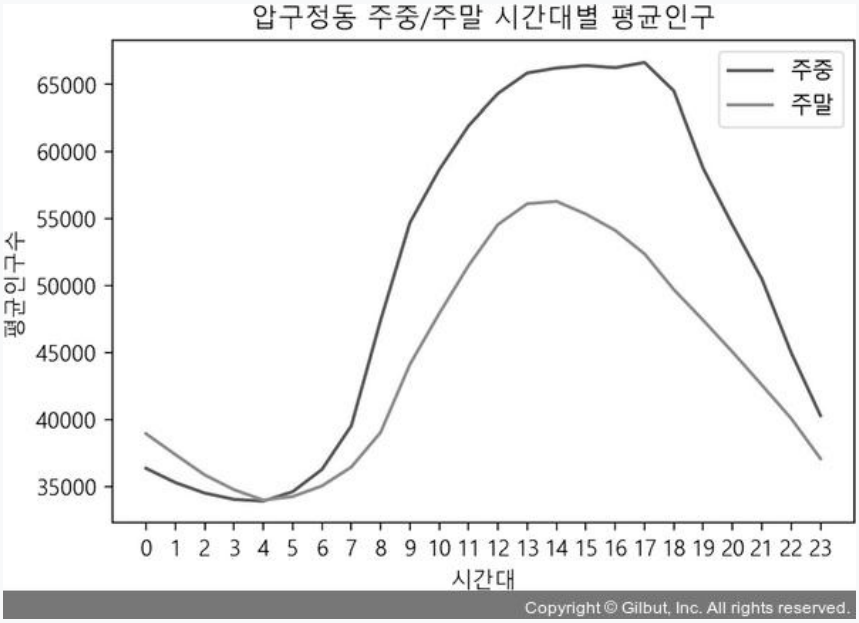
```
data, code_data = file_open()

input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)

# analysis1(dong_name = input_name, dong_code = input_code)
analysis2(dong_name = input_name, dong_code = input_code)
```

실행결과

```
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
압구정동 - 11680545 을(를) 분석합니다!
```



`analysis1()` 함수를 호출하는 부분은 주석 처리하고 `analysis2()` 함수만 실행합니다. 앞으로도 함수를 추가할 때마다 이전 함수는 주석으로 처리합니다. 그리고 모든 함수가 완성되면 함수 호출 부분을 정리하겠습니다.

코드를 실행하니 압구정동의 주중/주말 그래프가 그려집니다. 이차원 리스트로 인구 데이터를 넘겨도 그래프가 잘 그려지고 범례도 잘 표시됩니다. 완성된 코드를 함수로 재구성하는 것이 어렵지 않죠?

하위 목표 3 - 남녀 시간대별 인구 분석하기

하위 목표 3도 함수로 재구성해 봅시다. 성별 인구를 비교하는 그래프를 그려야 하는데, 주중/주말 인구에서 남녀 인구로 변경됐을 뿐이라서 앞에서 작성한 `analysis2()` 함수와 비슷합니다. 직접 수정해 본 후 책에 있는 코드와 비교해 보세요(변경 전 코드는 ' 11.2.4 하위 목표 3 - 남녀 시간대별 평균인구 분석하기'에 있습니다).

```
def analysis3(dong_name, dong_code):
    # 5.1 입력된 행정동의 남녀 시간대별 평균인구 구하기
    male = [0 for i in range(24)]
    female = [0 for i in range(24)]

    for row in data:
        if row[2] == dong_code:
            time = int(row[1])
            male[time] += sum(row[4:18])
            female[time] += sum(row[18:32])
    male = [m/31 for m in male]
    female = [f/31 for f in female]

    # 5.2 5.1에서 구한 평균인구 리스트로 꺾은선 그래프 그리기
    data_set = [male, female]
    labels = ['남성', '여성']
    title = dong_name + ' 남녀 시간대별 평균인구'
    graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

남녀 시간대별 인구를 구하는 부분은 그대로 `analysis3()` 함수 안에 넣고, 마지막에 그래프 그리는 부분만 `graph_plot()` 함수를 호출하도록 바꿉니다. 남성 인구 리스트 `male`과 여성 인구 리스트 `female`을 하나로 묶어 이차원 리스트 `data_set`을 만듭니다. 그래프의 순서대로 라벨을 리스트로 만들고 그래프 제목도 넣어서 `graph_plot()` 함수에 인자로 넘깁니다.

프로그램에 `analysis3()` 함수를 호출하는 부분을 추가합니다.

함수 호출 부분

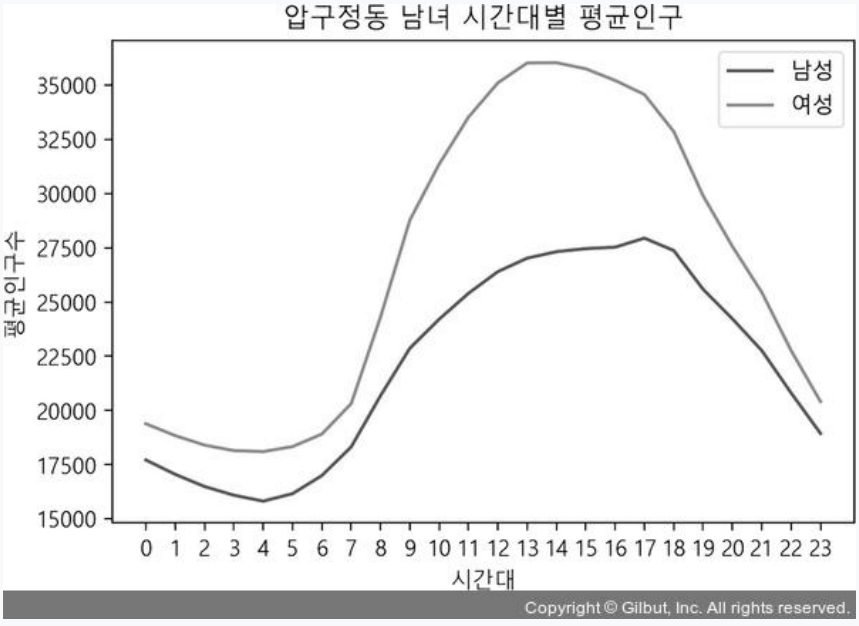
```
data, code_data = file_open()

input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)

# analysis1(dong_name = input_name, dong_code = input_code)
# analysis2(dong_name = input_name, dong_code = input_code)
analysis3(dong_name = input_name, dong_code = input_code)
```

실행결과

```
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
압구정동 - 11680545 을(를) 분석합니다!
```



남녀 시간대별 평균인구 비교 그래프도 잘 그려집니다.

하위 목표 4 - 다른 지역과 인구 비교 분석하기

마지막으로 하위 목표 4도 함수로 만들어 봅시다. 하위 목표 4는 하위 목표 1~3과 다르게 한 단계가 더 있습니다. 비교할 행정동을 입력받는 부분이죠. 이 부분은 앞서 만든 `dong_search()` 함수를 사용하면 됩니다. 그래프를 그리는 부분은 `graph_plot()` 함수를 호출하도록 변경하고요.

각 행정동의 평균인구를 구하는 부분을 봅시다. 이 부분은 하위 목표 1~3처럼 그대로 작성하면 될까요? 사실 평균인구를 구하는 함수는 이미 정의했습니다. 바로 `analysis1()` 함수입니다. 따라서 `analysis4()` 함수에서는 `analysis1()` 함수를 호출해 두 지역의 평균인구 리스트를 각각 구하면 됩니다. 그런데 `analysis1()`의 내용 전체가 다 필요한 건 아닙니다. 입력된 행정동의 평균인구를 구하는 부분만 필요하고, 그래프를 그리는 `graph_plot()` 함수를 호출하는 부분은 필요 없습니다. 따라서 그래프 그리는 부분을 선택적으로 사용하기 위해 `analysis1()` 함수를 다음과 같이 수정해야 합니다.

```
# graph_yn 매개변수 추가: 그래프 그리기가 필요하면 y, 필요하지 않으면 n
def analysis1(dong_name, dong_code, graph_yn = 'y'):
    population = [0 for i in range(24)]
    for row in data:
        if row[2] == dong_code:
            time, p = row[1], row[3]
            population[time] += p
    population = [p/31 for p in population]

    # 그래프를 그리지 않는 경우 인구 리스트만 반환하고 종료
    if graph_yn == 'n':
        return population

    population = [population]
    labels = ['평균인구']
    title = dong_name + ' 시간대별 평균인구'
    graph_plot(popu_list = population, label_list = labels, graph_title = title)
```

`analysis1()` 함수에 그래프를 그릴지 안 그릴지 전달받는 매개변수 `graph_yn`을 추가합니다. `analysis1()` 함수가 호출될 때 `graph_yn`에 값을 따로 입력하지 않으면 기본값 `y`로 간주하고 진행합니다. 그렇지 않고 `graph_yn`에 `n`이 전달된다면 그래프를 그리기 전에 인구 데이터 `population`을 반환하고 `analysis1()` 함수를 종료합니다. 조건문으로 그래프 그리는 부분을 선택할 수 있게 했습니다.

그럼 `analysis1()` 함수를 활용하도록 `analysis4()`를 작성해 봅시다(변경 전 코드는 '11.2.5 하위 목표 4 - 다른 지역과 인구 비교 분석하기'에 있습니다).

```
def analysis4(dong_name1, dong_code1):
    # 6.1 비교할 지역을 입력받아 행정동코드 알아내기
    dong_name2 = input('비교할 행정동을입력하세요. --> ')
    dong_code2 = dong_search(dong_name2)

    # 6.2 두 지역의 시간대별 평균인구 구하기
    population1 = analysis1(dong_name = dong_name1, dong_code = dong_code1, graph_yn = 'n')
    population2 = analysis1(dong_name = dong_name2, dong_code = dong_code2, graph_yn = 'n')

    # 6.3 6.2에서 구한 평균인구 리스트로 꺾은선 그래프 그리기
    data_set = [population1, population2]
    labels = [dong_name1, dong_name2]
    title = dong_name1 + '과 ' + dong_name2 + ' 시간대별 평균인구'
    graph_plot(popu_list = data_set, label_list = labels, graph_title = title)
```

미리 정의한 함수를 활용하니 기능에 비해 코드가 매우 간결합니다. 앞서 `analysis1()~ analysis3()` 함수는 매개변수가 `dong_name`, `dong_code`였습니다. 하지만 `analysis4()` 함수에서는 비교할 지역이 2곳이므로 매개변수인 핫플레이스가 있는 행정동을 `dong_name1`, `dong_code1`로 변경하고, 비교할 지역은 지역변수 `dong_name2`, `dong_code2`로 설정했습니다. 그리고 `analysis1()`의 결과로 반환되는 두 지역의 인구 리스트를 각각 변수 `population1`, `population2`에 저장합니다. `analysis1()` 함수를 호출할 때 `graph_yn = 'n'`을 추가하는 것도 잊지 마세요.

`population1`, `population2`로 그래프를 그립니다. 인구 데이터를 이차원 리스트로 만들고, 그래프 라벨도 지역 순서에 맞게 리스트에 넣어 `graph_plot()` 함수를 호출합니다.

프로그램에 `analysis4()` 함수를 호출하는 부분을 추가하고 `analysis4()`를 실행해 봅시다.

함수 호출 부분

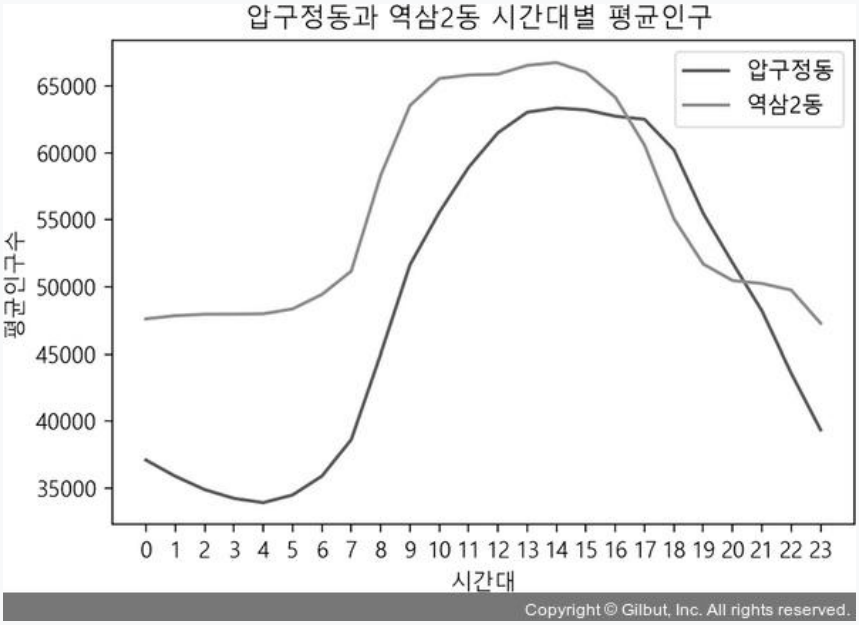
```
data, code_data = file_open()

input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)

# analysis1(dong_name = input_name, dong_code = input_code)
# analysis2(dong_name = input_name, dong_code = input_code)
# analysis3(dong_name = input_name, dong_code = input_code)
analysis4(dong_name1 = input_name, dong_code1 = input_code)
```

실행결과

```
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
압구정동 - 11680545 을(를) 분석합니다!
비교할 행정동을 입력하세요. --> 역삼2동
역삼2동 - 11680650 을(를) 분석합니다!
```



Copyright © Gilbut, Inc. All rights reserved.

함수 호출하기

이번에는 함수 호출 부분을 수정해 봅시다. 대부분 프로그램에서는 사용자가 원하는 것을 보여 주기 위해 사용자에게 입력을 받고 입력에 따른 결과를 출력합니다. 예를 들어, 뉴스에서 정치 부분을 선택하면 정치 관련 뉴스만, 경제 부분을 선택하면 경제 관련 뉴스만 보여 줍니다. 그리고 메신저에서도 친구를 누르면 연락할 수 있는 친구 목록이, 채팅을 누르면 현재 열려 있는 채팅창 목록이 보입니다.

이처럼 핫플레이스 프로젝트도 사용자가 보기를 원하는 항목의 번호를 입력받고, 이에 해당하는 그래프만 보여 줄 수 있습니다. 1번부터 4번까지 선택지를 주고, 숫자를 입력받아 해당 숫자에 해당하는 기능만 실행해 봅시다. 예를 들어, 1을 입력하면 하위 목표 1에 해당하는 그래프를, 3을 입력하면 하위 목표 3에 해당하는 그래프를 출력하는 것이죠.

함수 호출 부분

```
# 필요한 파일 읽기
data, code_data = file_open()

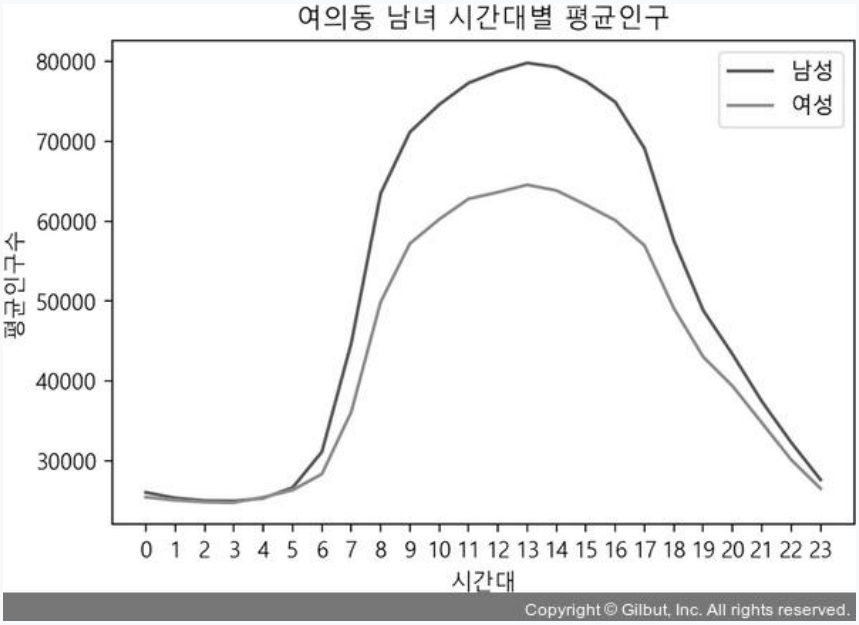
print('핫플레이스는 언제 가야 가장 덜 붐빌까?')

# 핫플레이스가 있는 행정동 입력받기
input_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
input_code = dong_search(input_name)
print('1. 시간대별 평균인구는 어떻게 될까?')
print('2. 주중에 가야 할까, 주말에 가야 할까?')
print('3. 남성이 많을까, 여성이 많을까?')
print('4. 나에게 익숙한 지역과 비교해 얼마나 붐빌까?')

selection = int(input('분석하고 싶은 내용의 번호를 입력해 주세요. -->'))
if selection == 1:
    analysis1(dong_name = input_name, dong_code = input_code)
elif selection == 2:
    analysis2(dong_name = input_name, dong_code = input_code)
elif selection == 3:
    analysis3(dong_name = input_name, dong_code = input_code)
elif selection == 4:
    analysis4(dong_name1 = input_name, dong_code1 = input_code)
else:
    print('잘못된 입력입니다.')
```

실행결과

```
핫플레이스는 언제 가야 가장 덜 붐빌까?
핫플레이스가 위치한 행정동을 입력하세요. --> 여의동
여의동 - 11560540 을(를) 분석합니다!
1. 시간대별 평균인구는 어떻게 될까?
2. 주중에 가야 할까, 주말에 가야 할까?
3. 남성이 많을까, 여성이 많을까?
4. 나에게 익숙한 지역과 비교해 얼마나 붐빌까?
분석하고 싶은 내용의 번호를 입력해 주세요. --> 3
```



프로젝트 주제와 각각의 하위 목표를 먼저 출력하고, `selection` 변수에 분석하려는 번호를 입력받습니다. `selection`이 1~4일 때 각각 `analysis1()~analysis4()` 함수를 호출하고, 그 외 값을 입력하면 잘못된 입력이라고 출력합니다. 조건에 맞는 함수를 실행하도록 `if-elif-else` 문을 사용합니다.

지금까지 11장의 핫플레이스 프로젝트를 함수를 활용해 재구성해 봤습니다. 결과는 동일하지만 내부 코드는 훨씬 더 간결하고 직관적인 방향으로 수정됐습니다. 이처럼 프로그램의 기능이 많아지고 코드가 길어질 때 함수를 활용하면 효율적인 프로그램을 작성할 수 있습니다.

13

마무리

1 사용자 정의 함수 정의하기

형식

```
def 함수명(매개변수1, 매개변수2):
    함수 내용
    return 반환값
```

2 사용자 정의 함수 호출하기

형식

```
함수명(매개변수1 = 인자1, 매개변수2 = 인자2)
# 또는
함수명(인자1, 인자2)
```

사용자 정의 함수를 호출할 때는 매개변수에 직접 인자를 대입하거나 매개변수를 생략하고 인자로만 호출하는 두 가지 방법을 사용할 수 있다. 매개변수를 생략하고 인자만 전달하는 경우, 함수를 정의할 때 설정한 매개변수의 순서대로 인자가 차례대로 대입된다.

3 사용자 정의 함수의 매개변수에 기본값 설정하기

형식

```
def 함수명(매개변수1 = 기본값1, 매개변수2 = 기본값2):
    함수 내용
    return 반환값
```

Self Check

1 구구단 프로그램을 작성하려고 합니다. 숫자를 입력받으면 해당 단 전체를 출력하는 `gugudan()` 함수를 작성하세요. 예를 들어, `gugudan(9)`를 실행하면 결과는 다음과 같습니다.

```
gugudan(9)
```

```
실행결과
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

2 1번에서 작성한 `gugudan()` 함수를 활용해 구구단을 2단부터 9단까지 출력하는 프로그램을 작성하세요. 각 단이 시작될 때 제시된 실행결과처럼 몇 단인지 출력하는 기능도 추가하세요.

실행결과

```
2 단 --------
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
...
9 단 --------
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

3 점수를 입력받아 등급을 산출하는 `cal_grade()` 함수를 작성하려고 합니다. 80점 이상은 A, 40점 이상 80점 미만은 B, 40점 미만은 C로 출력하되 다음 결과처럼 출력되게 하세요.

```
# cal_grade() 함수 정의 부분

s = int(input('점수 : '))
print('등급 :', cal_grade(s))
```

실행결과

```
점수 : 85
등급 : A
```

정답 및 해설